Chapter 3:

Memory and Storage System

1. What are characteristics of memory devices?

The key characteristics of memory devices or memory system are as follows:

a) Location
b) Capacity
c) Unit of Transfer
d) Access Method
e) Performance
f) Physical type
g) Physical characteristics
h) Organization

a. **Location:**

It deals with the location of the memory device in the computer system. There are three possible locations:

- CPU : This is often in the form of CPU registers and small amount of cache
- Internal or main: This is the main memory like RAM or ROM. The CPU can directly access the main memory.
- External or secondary: It comprises of secondary storage devices like hard disks, magnetic tapes. The CPU doesn't access these devices directly. It uses device controllers to access secondary storage devices.

b) **Capacity:**

The capacity of any memory device is expressed in terms of: i)word size ii)Number of words

- **Word size:** Words are expressed in bytes (8 bits). A word can however mean any number of bytes. Commonly used word sizes are 1 byte (8 bits), 2bytes (16 bits) and 4 bytes (32 bits).

- **Number of words:** This specifies the number of words available in the particular memory device. For example, if a memory device is given as 4K x 16.This means the device has a word size of 16 bits and a total of 4096(4K) words in memory.

c) **Unit of Transfer:**

It is the maximum number of bits that can be read or written into the memory at a time. In case of main memory, it is mostly equal to word size. In case of external memory, unit of transfer is not limited to the word size; it is often larger and is referred to as blocks.

d) **Access Methods:**

It is a fundamental characteristic of memory devices. It is the sequence or order in which memory can be accessed. There are three types of access methods:

- **Random Access:** If storage locations in a particular memory device can be accessed in any order and access time is independent of the memory location being accessed. Such memory devices are said to have a random access mechanism. RAM (Random Access Memory) IC's use this access method.
- **Serial Access:** If memory locations can be accessed only in a certain predetermined sequence, this access method is called serial access. Magnetic Tapes, CD-ROMs employ serial access methods.
- **Semi random Access:** Memory devices such as Magnetic Hard disks use this access method. Here each track has a read/write head thus each track can be accessed randomly but access within each track is restricted to a serial access.

e) **Performance:** The performance of the memory system is determined using three parameters

- **Access Time:** In random access memories, it is the time taken by memory to complete the read/write operation from the instant that an address is sent to the memory. For non-random access memories, it is the time taken to position the read write head at the desired location. Access time is widely used to measure performance of memory devices.
- **Memory cycle time:** It is defined only for Random Access Memories and is the sum of the access time and the additional time required before the second access can commence.

- **Transfer rate:** It is defined as the rate at which data can be transferred into or out of a memory unit.

f) **Physical type**: Memory devices can be either semiconductor memory (like RAM) or magnetic surface memory (like Hard disks).

g) **Physical Characteristics:**

- **Volatile/Non- Volatile:** If a memory devices continues hold data even if power is turned off. The memory device is non-volatile else it is volatile.

h) **Organization:**

- **Erasable/Non-erasable:** The memories in which data once programmed cannot be erased are called Non-erasable memories. Memory devices in which data in the memory can be erased is called erasable memory.

  E.g. RAM(erasable), ROM(non-erasable).

2) Explain in details Memory hierarchy with examples.

- The memory system is a hierarchy of storage devices with different capacities, costs, and access times.
- The idea centers on a fundamental property of computer programs known as locality. Programs with good locality tend to access the same set of data items over and over again, or they tend to access sets of nearby data items. Programs with good locality tend to access more data items from the upper levels of the memory hierarchy than programs with poor locality, and thus run faster. Figure 1 shows a typical memory hierarchy with relationship between the cost per bit, the access time and the hierarchy.

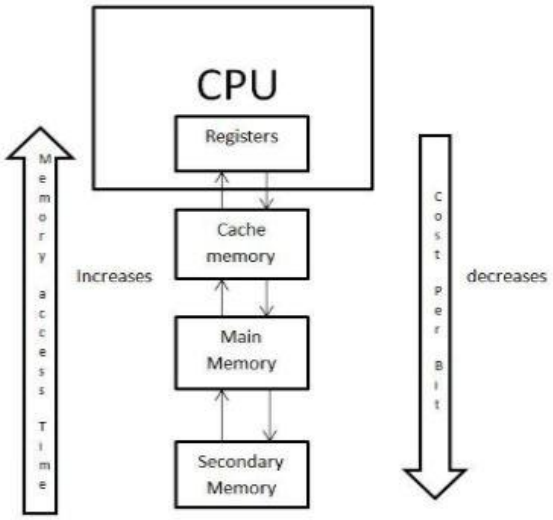


Figure 1

- Memory hierarchies work because well-written programs tend to access the storage at any particular level more frequently than they access the storage at the next lower level. So the storage at the next level can have a larger access time, and thus will be larger in size and cheaper per bit.
- The overall effect is a large pool of memory that costs as much as the cheap storage near the bottom of the hierarchy, but that serves data to programs at almost the rate of the fast storage near the top of the hierarchy.

1. Registers

- CPU **registers** are at the top most level of this hierarchy, they hold the most frequently used data. They are very limited in number and are the **fastest**.
- They are often used by the CPU and the ALU for performing arithmetic and logical operations, for temporary storage of data.

2. Cache

- The very next level consists of small, fast **cache memories** near the CPU. They act as
- staging areas for a subset of the data and instructions stored in the relatively slow **main memory**.
- There are often two or more levels of cache as well. The cache at the top most level after the registers is the primary cache. Others are secondary caches.

- Many a times there is cache present on board with the CPU along with other levels that are outside the chip.

3. Main Memory:

- The next level is the main memory, it stages data stored on large, slow disks often called hard disks. These hard disks are also called secondary memory, which are the last level in the hierarchy. The main memory is also called primary memory.
- The secondary memory often serves as staging areas for data stored on the disks or tapes of other machines connected by networks.

Performance Considerations: (Access Time vs cost vs size)

- A programmer needs to understand the memory hierarchy because it has a big impact on the performance of his applications.
- For example:

  If the data your program needs are stored in a CPU register, then they can be accessed in zero cycles during the execution of the instruction. If stored in a cache, 1 to 30 cycles. If stored in main memory, 50 to 200 cycles. And if stored in disk tens of millions of cycles. This is obviously delay being introduced.

- On the other hand, the **cost per bit** is the least for the secondary memory, which is more at the primary memory level. This increases till at the register level where it is the maximum.
- Similarly, memory sizes are smallest at the register level. This increases to a few KBs in the cache, few hundreds of MBs or even GBs nowadays in the main memory, to hundreds of GB's in the secondary memory.
- Thus there is a tradeoff between cost, access time, size and all this needs to be taken into account when designing the memory hierarchy for a computer system.

3. Discuss virtual memory?

Virtual memory, simply put is when the hard disk is used to provide an extension to main memory. In essence, virtual memory is a facility that allows programs to address memory from a logical point of view, without regard to the amount of main memory physically available.

When virtual memory is used, the address fields of machine instructions contain virtual addresses. For reads to and writes from main memory, a hardware memory management unit (MMU) translates each virtual address into a physical address in main memory. The advantage of using such a scheme is that it is possible for a process to be larger than all of main memory. One of the most fundamental restrictions in programming has been lifted.

The process of address translation is shown in Figure 2 which shows the Virtual Memory organization as well.

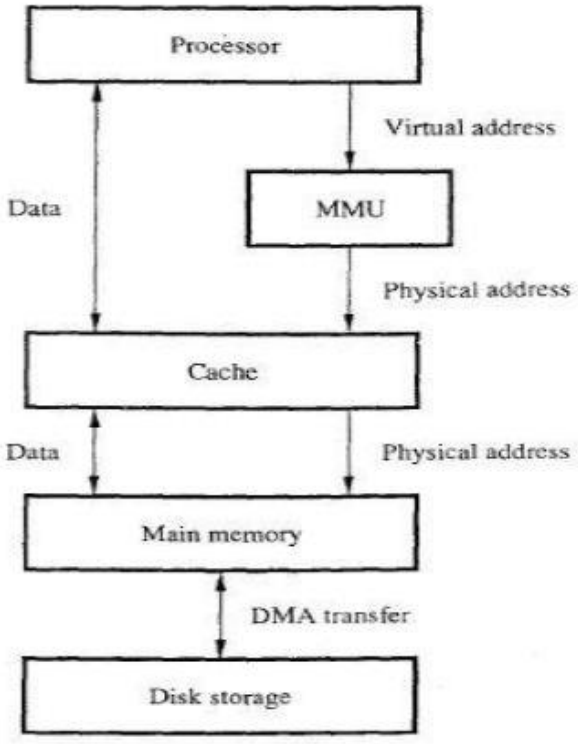


Figure 2

Without demand paging (this is the practice of getting pages from the secondary memory into the main memory as and when required. Programs are considered to be divided into **fixed length/fixed size** pages) and virtual memory, a programmer must be acutely aware of how much memory is available.

If the program being written is too large, the programmer must devise ways to structure the program into pieces that can be loaded one at a time. With virtual memory scheme and demand paging as specified above, that job is left to the OS and the hardware. As far as the programmer is

concerned, he or she is dealing with a huge memory, the size associated with disk storage. Because a process executes only in main memory, that memory is referred to as real memory. But a programmer or user perceives a much larger memory—that which is allocated on the disk. This latter is therefore referred to as **virtual memory**.

Virtual memory allows for very effective multiprogramming and relieves the user of the unnecessarily tight constraints of main memory.

4. Explain Cache Memory and describe Cache mapping technique.

Cache Memory: -

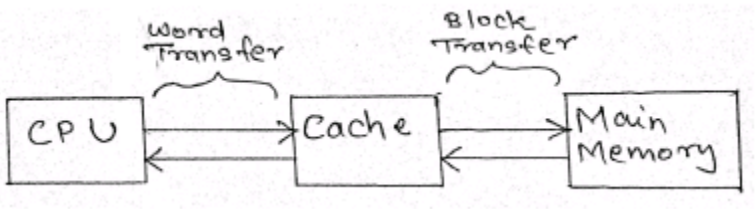
1) Cache Memory is very high speed memory used to increase the speed of program by making current program & data available to the CPU at a rapid rate.

2) Access time to cache memory is less compared to main memory. It contains a copy of potions of the main memory.

3) When CPU attempts to read a word from main memory, check is made to determine if the word is in cache. It so, then word is delivered form cache.

4) If word is not there in cache then a block of main memory consisting some word along with that word, is read into cache and the required word is delivered to CPU. This is called “Principle of Locality of Reference”.

5) During a miss if there are no empty blocks in the cache, then some replacement policies such as FIFO, LRU, LFU, etc. are used.



Cache Mapping Technique: -

The different Cache mapping technique are as follows:-

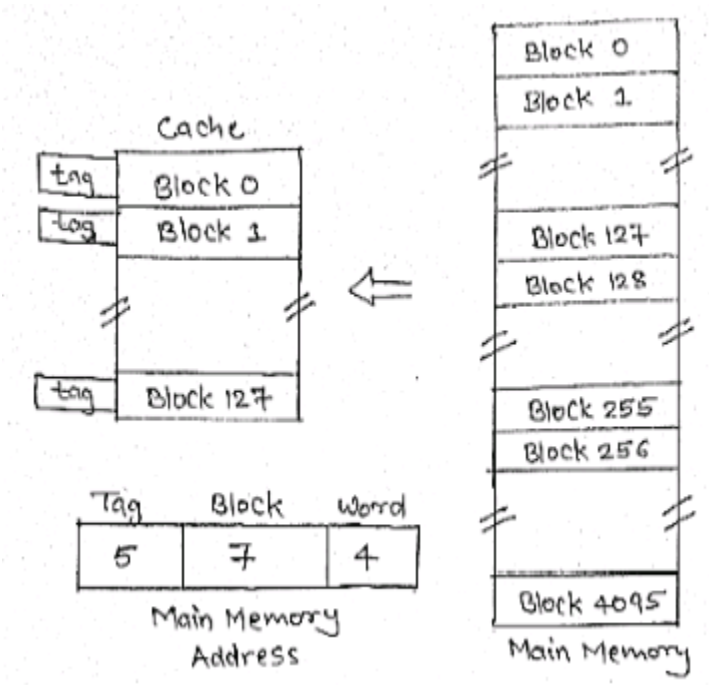
1) Direct Mapping

2) Associative Mapping

3) Set Associative Mapping

Consider a cache consisting of 128 blocks of 16 words each, for total of 2048(2K) works and assume that the main memory is addressable by 16 bit address. Main memory is 64K which will be viewed as 4K blocks of 16 works each.

(1) Direct Mapping: -



1) The simplest way to determine cache locations in which store Memory blocks is direct Mapping technique.

2) In this block J of the main memory maps on to block J modulo 128 of the cache. Thus main memory blocks 0,128,256,….is loaded into cache is stored at block 0. Block 1,129,257,….are stored at block 1 and so on.

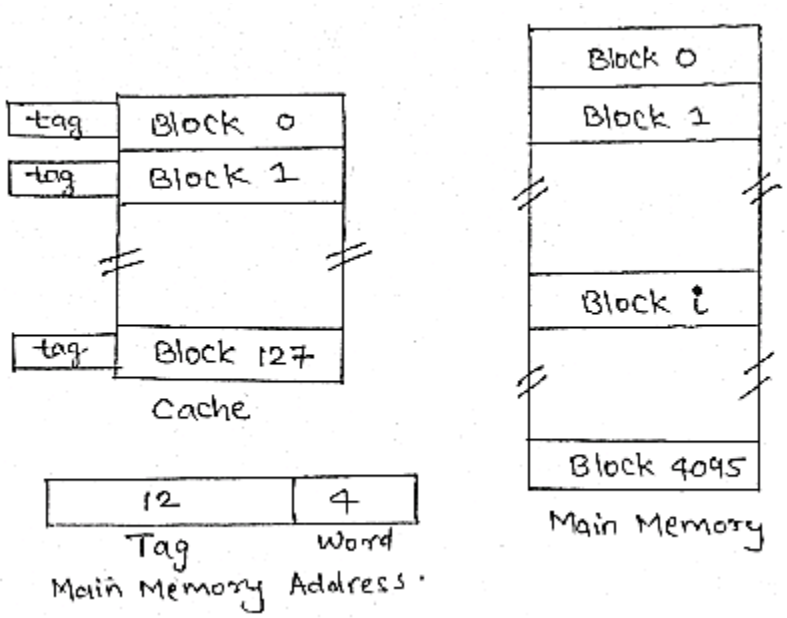
3) Placement of a block in the cache is determined from memory address. Memory address is divided into 3 fields, the lower 4-bits selects one of the 16 words in a block.

4) When new block enters the cache, the 7-bit cache block field determines the cache positions in which this block must be stored.

5) The higher order 5-bits of the memory address of the block are stored in 5 tag bits associated with its location in cache. They identify which of the 32 blocks that are mapped into this cache position are currently resident in the cache.

6) It is easy to implement, but not Flexible

(2) Associative Mapping: -



1) This is more flexible mapping method, in which main memory block can be placed into any cache block position.

2) In this, 12 tag bits are required to identify a memory block when it is resident in the cache.

3) The tag bits of an address recevied from the processor are compared to the tag bits of each block of the cache to see, if the desired block is present. This is known as Associative Mapping technique.

4) Cost of an associated mapped cache is higher than the cost of direct-mapped because of the need to search all 128 tag patterns to determine whether a block is in cache. This is known as associative search.

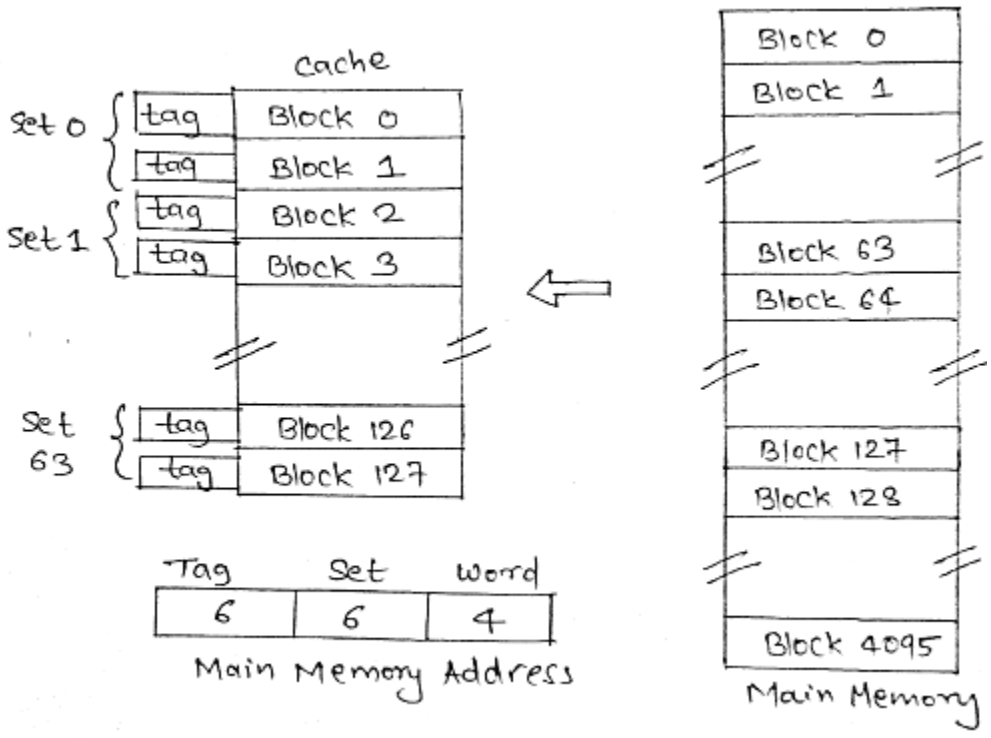
(3) Set-Associated Mapping: -

1) It is the combination of direct and associative mapping technique.

2) Cache blocks are grouped into sets and mapping allow block of main memory reside into any block of a specific set. Hence contention problem of direct mapping is eased , at the same time , hardware cost is reduced by decreasing the size of associative search.

3) For a cache with two blocks per set. In this case, memory block 0, 64, 128,…..,4032 map into cache set 0 and they can occupy any two block within this set.

4) Having 64 sets means that the 6 bit set field of the address determines which set of the cache might contain the desired block. The tag bits of address must be associatively compared to the tags of the two blocks of the set to check if desired block is present. This is two way associative search.



5. Explain set associative and associative cache mapping techniques.

Set-Associative Mapping

1. The direct mapped cache can hold maximum 256 words.

2. The set-associative cache is an imported version of direct mapped cache organization, where multiple of 256 words can be stored, but with increased cost.
3. In set-associative cache memory two or more words can be stored under the same index address. Each data word is stored together with its tag.
4. The number of tag-data words under an index is said to form a set. If k number of words with their associated tags (i.e. set site = k) are stored under an index of cache,. Then the cache memory is called k-way set-associative.
5. A 2-way set-associative memory is shown in figure, where two data words together with two tag values are stored in a single index address.
6. For 2-way set-associative cache, the word length is 2(8 + 8) = 32 bits, since each tag requires 8 bits and each data word requires 8 bits. So, the cache memory size is now converted to 256 x 32. This means that the cache memory can hold 512 words of the main memory.
7. When the CPU generates a memory address of 16.bit, the index value of 8-bits is used to access the cache. Then the tag field of CPU address is compared with both tags under the selected index of the 2-way set associative cache memory for bits. The comparison of tags in the set of cache memory is done using the associative search technique, which is why the mapping technique is called set-associative mapping.
8. In this case, since multiple number of words is stored under a common index value the hit ratio improves compared to previous two techniques.
9. If the set is full and a miss occurs in a set-associative cache, then one of the stored tag-data pairs must he replaced with a new pair value from the main memory.



Figure: 2-way set associative cache memory

Merits of set-associative cache:

The cache memory has highest hit ratio compared to other two cache memories.

Demerits of set-associative cache:

This is the most expensive memory. The cost increases as set size increases.

6. Describe what are the features of cache design?

Cache Size

- There are several motivations for minimizing the cache size. The larger the cache, the greater the number of gates involved in addressing the cache is needed.
- The result is that larger caches end up being slightly slower than small ones. The available chip and board area also limits cache size.
- Because the performance of the cache is very sensitive to the nature of the workload, it is impossible to arrive at a single optimum cache size.

Mapping Function

- Mapping functions are used as a way to decide which main memory block occupies which line of cache. As there are less lines of cache than there are main memory blocks, an algorithm is needed to decide this.
- Three techniques are used, namely direct, and associative and set associative, which dictate the organization of the cache.

Direct:

- This is the simplest form of mapping. One block from main memory maps into only one possible line of cache memory.
- As there are more blocks of main memory than there are lines of cache, many blocks in main memory can map to the same line in cache memory.

Associative:

- This type of mapping overcomes the disadvantage of direct mapping by permitting each main memory block to be loaded into any line of cache.
- To do so, the cache control logic interprets a memory address as a tag and a word field. The tag uniquely identifies a block in main memory.
- The primary disadvantage of this method is that to find out whether a particular block is in cache, all cache lines would have to be examined. Using this method, replacement algorithms are required to maximize its potential.

Set Associative:

- This type of mapping is designed to utilize the strengths of the previous two mappings, while minimizing the disadvantages.
- The cache is divided into a number of sets containing an equal number of lines.
- Each block in main memory maps into one set in cache memory similar to that of direct mapping.
- Within the set, the cache acts as associative mapping where a block can occupy any line within that set. Replacement algorithms may be used within the set

Replacement Algorithms

For direct mapping where there is only one possible line for a block of memory, no replacement algorithm is needed. For associative and set associative mapping, however, an algorithm is needed. For maximum speed, this algorithm is implemented in the hardware. Four of the most common algorithms are:

1) Least recently used this replaces the candidate line in cache memory that has been there the longest with no reference to it.

2) First in first out this replaces the candidate line in the cache that has been there the longest.

3) Least frequently used this replaces the candidate line in the cache that has had the fewest references.

4) Random replacement this algorithm randomly chooses a line to be replaced from among the candidate lines. Studies have shown that this yields only slightly inferior performance than other algorithms.

Write Policy

This is important because if changes were made to a line in cache memory, the appropriate changes should be made to the block in main memory before removing the line from the cache. The problems to contend with are more than one device may have access to main memory (I/O modules). If more than one processor on the same bus with its own cache is involved, the problem becomes more complex. Any change in either cache or main memory could invalidate the others.

The simplest technique is called “write through”. In using this technique, both main memory and cache are written to when a write operation is performed, ensuring that main memory is always valid. The main disadvantage of this technique is that it may generate substantial main memory traffic, causing a bottle neck and decreasing performance.

An alternative technique, known as “write back” minimizes main memory writes. Updates are made only in the cache. An update bit associated with the line is set. Main memory is updated when the line in cache gets replaces only if the update bit has been set. The problem with this technique is that all changes to main memory have to be made through the cache in order not to invalidate parts of main memory, which potentially may cause a bottle neck.

Line Size

When a block of data is retrieved from main memory and put into the cache, the desired word and a number of adjacent words are retrieved. As the block size increases from a very small size, the hit ratio will at first increase due to the principle of locality of reference, which says that words in the vicinity of a referenced word are more likely to be referenced in the near future. As the block

size increases, however, the hit ratio will decrease as the probability of reusing the new information becomes less than that of using the information that has been replaced.

Number of Caches

Two aspects of this are:

Multilevel Due to increased logic density, it has become possible to have a cache on the same chip as the processor. This increases execution time as less activity over an external bus is needed. Even though an on-chip cache exists, it is typically desirable to have an off-chip cache as well. This means that if a miss occurs on the level 1 cache (on-chip), instead of retrieving the data from the slower main memory, information may be retrieved from the level 2 cache, which, although slower than level 1 cache, is still appreciably faster than main memory. Some level 2 caches are stored on-chip and a level 3 cache has been implemented off-chip. Unified/Split Two types of words exist that are stored in cache, namely data and instruction. It has become common to split the cache into two to separate these words.

Two Potential advantages to unified cache are:

A greater hit rate than split caches because the load between instruction and data fetches are balanced automatically.

Only one cache needs to be designed and implemented. The key advantage of the split cache design is that it eliminates contention for the cache between the instructions fetch/decode unit and the execution unit. This is important for designs that rely on pipelining of instructions

7. Explain virtual memory with reference to memory hierarchy segments and pages.

1. When a program does not fit into the main memory, the parts of the program that are currently not being executed are stored on the hard disk and are transferred to main memory as and when required. Thus simply put, Virtual memory is just the extension of the main memory in the disk where parts of very big programs that are currently being executed but cannot be stored completely in the main memory itself are stored.
2. This shows that the CPU no longer generates the physical address directly. This is translated to the physical address by the MMU which is then used to address the main memory as shown in Figure 6:

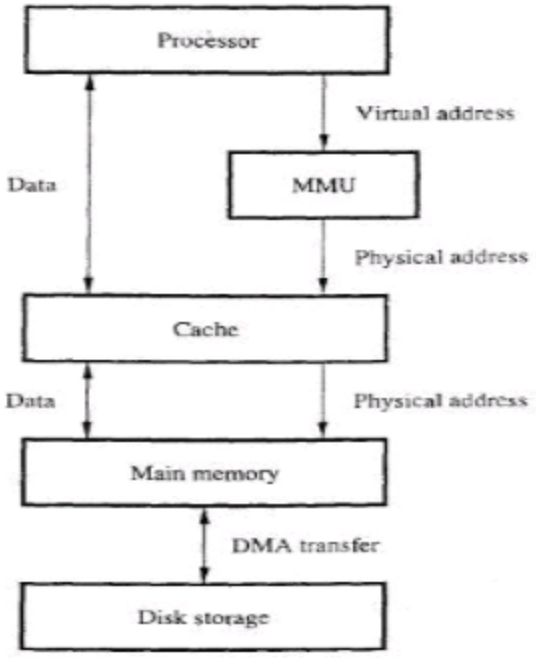


Figure 6

Paging:

1. Programs getting divided into fixed size partitions or pieces named pages. This led to the development of the important concept of virtual memory. With the use of paging, truly effective multiprogramming systems came into being. Virtual Memory uses **demand paging**, which simply means that each page of a process is brought in only when it is needed, that is, on demand. It is clear that we can make better use of memory by loading in just a few pages.

2. Then, if the program branches to an instruction on a page not in main memory, or if the program references data on a page not in memory, a **page fault** is triggered. This tells the OS to bring in the desired page.

3. The basic mechanism for reading a word from memory involves the translation of a virtual, or logical, address, consisting of page number and offset, into a physical address, consisting of frame number and offset, using a page table. Because the page table is of variable length, depending on the size of the process, we cannot expect to hold it in registers. Instead, it must be in main memory to be accessed.

4. When a particular process is running, a register (Page Table Register) holds the starting address of the page table for that process. The page number of a virtual address is used to index that table and look up the corresponding frame number. This is combined with the offset portion of the virtual address to produce the desired real address. Physical address generation from virtual addresses is shown in Figure 7.

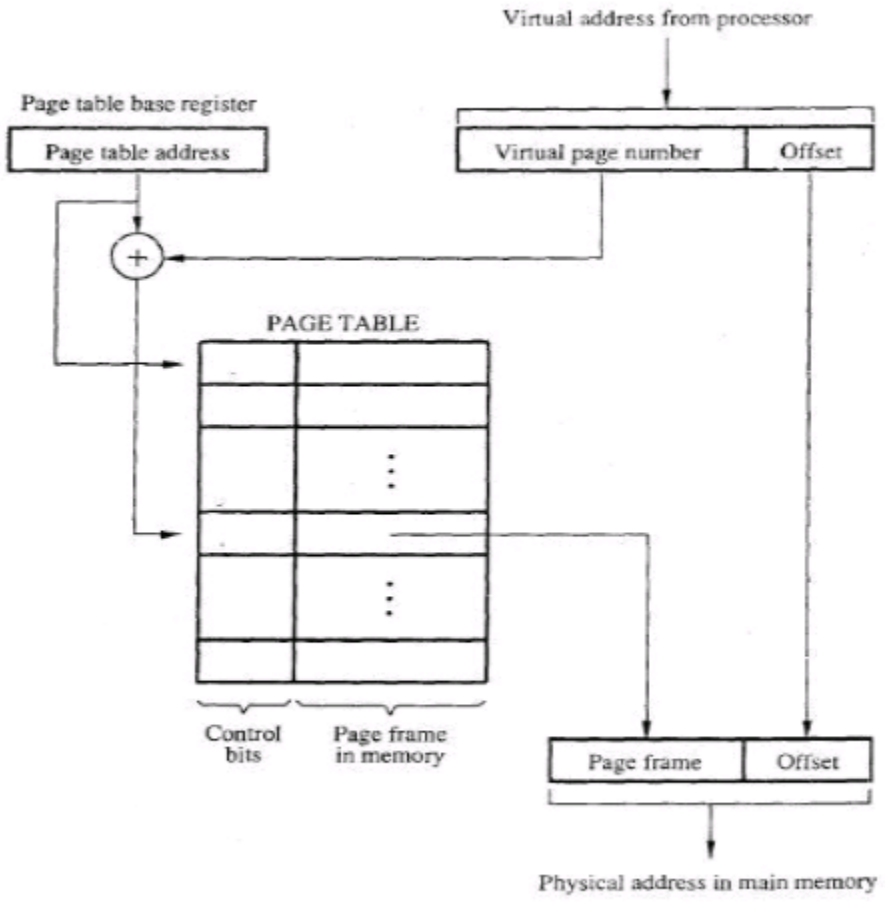


Figure 7

Segmentation:

- Paging was invisible to the programmer and serves the purpose of providing the programmer with a larger address space, segmentation is usually visible to the programmer and is provided as a convenience for organizing programs and data and as a means for associating privilege and protection attributes with instructions and data. Segments basically contain many pages.

- Thus Segmentation allows the programmer to view memory as consisting of multiple address spaces or segments. Segments are of variable, indeed dynamic, size while pages were fixed size fragments. OS will assign programs and data to different segments. There may be a number of program segments for various types of programs as well as a number of data segments. Each segment may be assigned access and usage rights. Memory references consist of a (segment number, offset) form of address.
- The address translation mechanism when segmentation and paging is used with paging is shown in Figure 8.

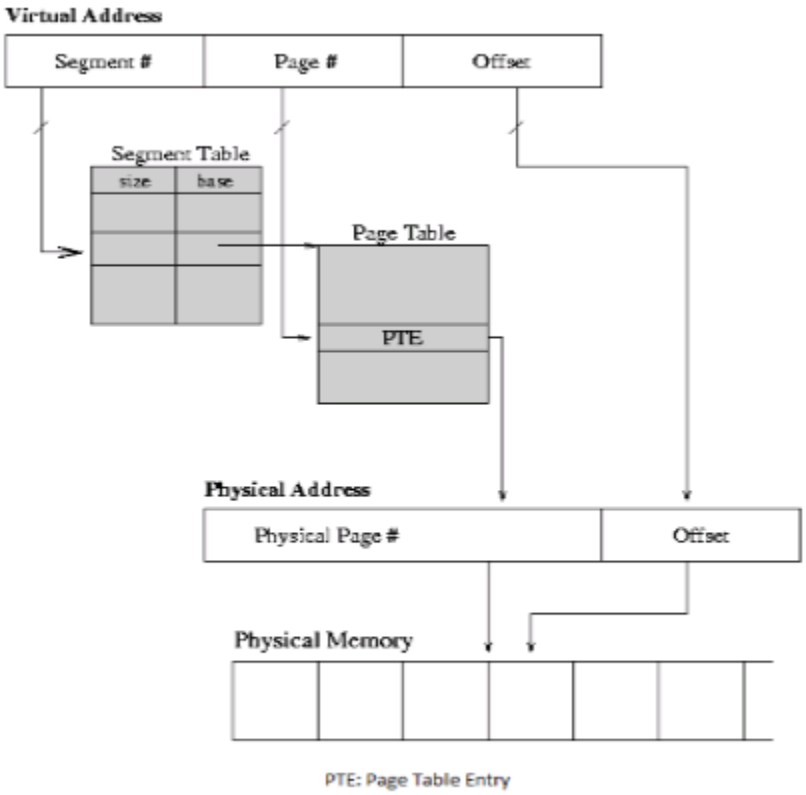


Figure 8

- The virtual address is now composed of three parts: The higher order bits give the segment number, the bits after that give the page number and the bits after that give the offset.
- Thus an extra Segment table is used. This along with the page number is used to index the page table.
- The process after that is similar to the address translation mechanism when paging was used.

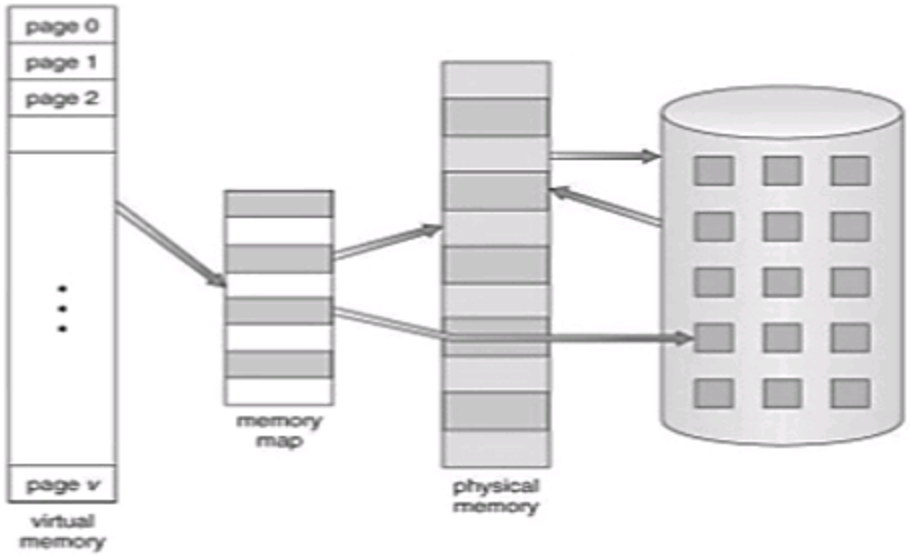
- Segmentation is optional and can be disabled.

8. Explain paging in detail. Describe how logical address is converted into physical address?

- Paging is one of the memory management schemes by which a computer stores and retrieves data from the secondary storage for use in main memory.
- In the paging memory-management scheme, the operating system retrieves data from secondary storage in same-size blocks called pages.
- Paging allows the physical address space of the process to be non contiguous
- Paging is to deal with external fragmentation problem. This is to allow the logical address space of a process to be non-contiguous, which makes the process to be allocated physical memory.
- Logical address space of a process can be non-contiguous; process is allocated physical memory.

Whenever the latter is available.

- Divide physical memory into fixed-sized blocks called frames (size is power of 2, between 512 bytes and 8192 bytes).
- Divide logical memory into blocks of same size called pages.
- Keep track of all free frames.
- To run a program of size n pages, need to find n free frames and load program.
- Set up a page table to translate logical to physical addresses.
- Internal fragmentation-allocated memory may be slightly larger than requested memory; this size difference is memory internal to a partition, but not being used
- The logical memory of the process is contiguous, but pages need not be allocated contiguously in memory.
- By dividing memory into fixed size pages, we can eliminate external fragmentation.
- Paging does not eliminate internal fragmentation



Address generated by CPU is divided into:

1. Page number (p) – used as an index into a page table which contains base address of each page in physical memory.

2. Page offset (d) – combined with base address to define the physical memory address that is sent to the memory unit.

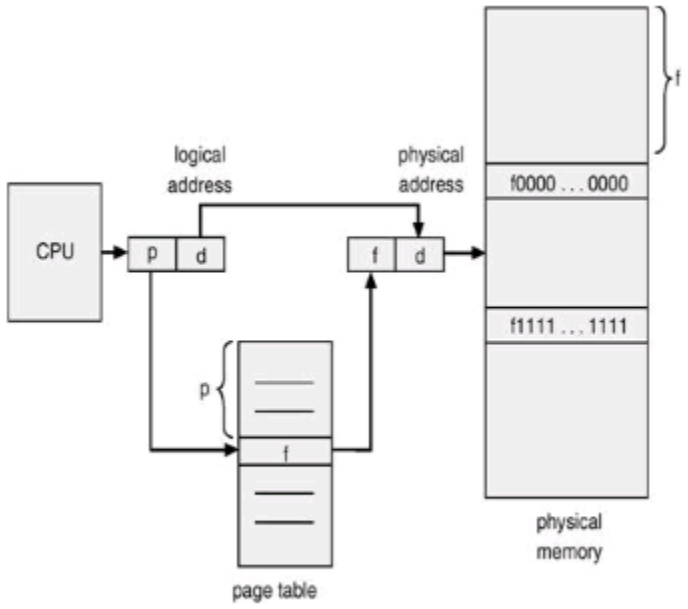
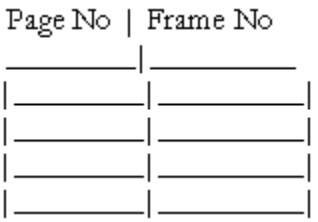


Fig. Address translation scheme

Implementation of page table:

- Page table is kept in main memory.
- Page-table base register (PTBR) points to the page table.
- Page-table length register (PTLR) indicates size of the page table.
- In this scheme every data/instruction access requires two memory accesses. One for the page table and one for the data/instruction.
- The two memory access problem can be solved by the use of a special fast-lookup hardware cache called associative registers or translation look-aside buffers (TLBs).
- Associative registers - parallel search



Address translation (A', A")

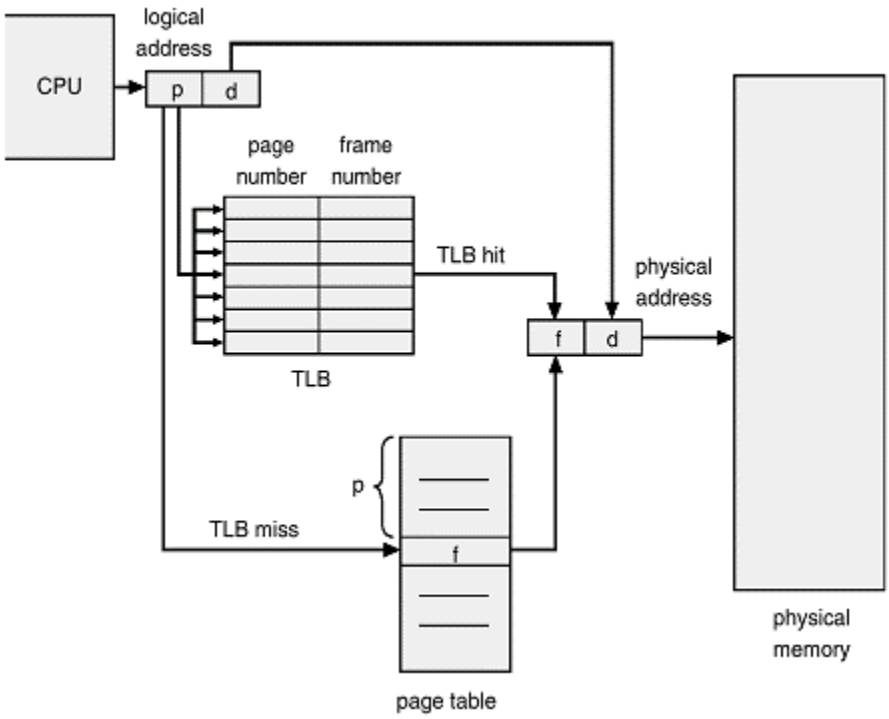
- If A' in associative register, get frame number out.
- Otherwise get frame number from page table in memory.
    - Hit ratio - percentage of times that a page number is found in the associative registers; ratio related to number of associative registers.
    - Effective Access Time (EAT)
    - associative lookup = e time units
    - memory cycle time = m time units
    - hit ratio = a

EAT = (m + e) a+ (2m + e) (1 - a) = 2m + e - ma

- Memory protection implemented by associating protection bits with each frame.
- Valid-invalid bit attached to each entry in the page table:

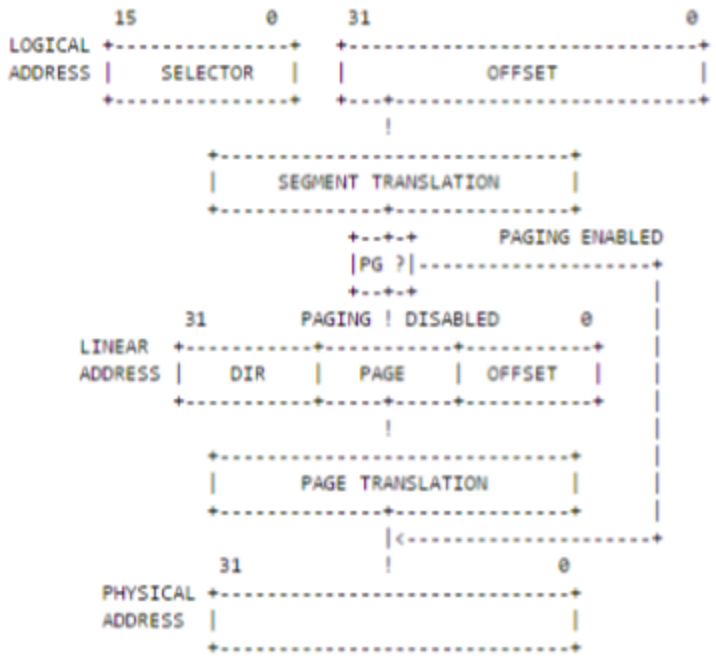
- ``valid" indicates that the associated page is in the process' logical address space, and is thus a legal page.
- ``invalid" indicates that the page is not in the process' logical address space.

- Write bit attached to each entry in the page table.
  - Pages which have not been written may be shared between processes
  - Do not need to be swapped - can be reloaded.



Conversion of logical address to physical address

- The concept of a logical address space that is bound to a separate physical address space is central to proper memory management.
  - Logical address - generated by the CPU; also referred to as virtual address.
  - Physical address - address seen by the memory unit.
- Logical and physical addresses are the same in compile-time and load-time address-binding schemes; logical (virtual) and physical addresses differ in execution-time address-binding scheme.

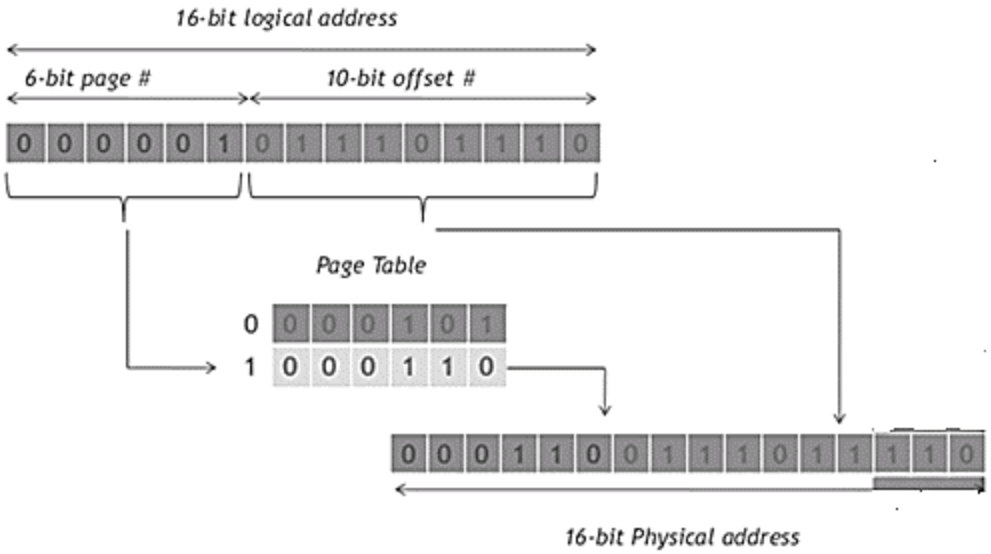


There are three steps to translate logical address to physical address

Step1: Find the index field from the segment selector and use the index field to locate the segment descriptor for the segment in global descriptor table (GDT)

Step 2: Test the access and limit the field of descriptor to make sure that the segment is accessible and the offset is within the limit of the segment

Step3: The base address of the segment will be obtained from the segment descriptor. Then the base address of the segment will be added to the offset to determine a linear address



9. Explain various high-speed memories such as interleaved memories and caches.

- The main purpose of introduction of a memory hierarchy was showing the CPU a single large memory space having a small access time. If the main memory of a computer is arranged as a collection of physically separate banks each with its own address buffer register (ABR) and data buffer register (DBR), memory access cab continue in more than one bank at a time.

Interleaved Memory:

- Interleaved memory is a design made to compensate for the relatively slow speed of dynamic random-access memory (DRAM).This is done by spreading memory addresses evenly across memory banks. Thus contiguous memory reads and writes are done using each memory bank in turn, resulting in higher memory throughputs due to reduced waiting for memory banks to become ready for desired operations.
- The idea of interleaved memory is shown in Figure 9 below:

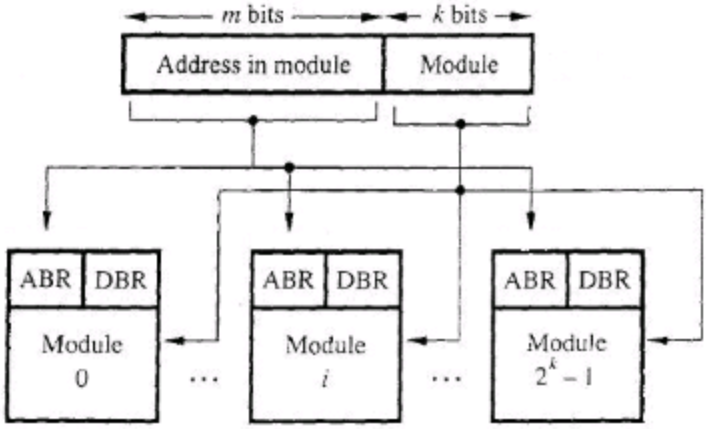


Figure 9

- As shown in Figure 9, the lower order k bits of the address are used to select the module (Memory bank) and higher order m bits give a unique memory location in the memory bank that is selected by the lower order k bits. Thus in this way consecutive memory locations are stored on different memory banks.
- Whenever requests to access consecutive memory locations are being made several memory banks are kept busy at any point in time. This results in faster access to a block of data in the memory and also results in higher overall utilization of the memory system as a whole. If k bits are allotted for selecting the bank as shown in the diagram, there have to be total 2k banks. This ensures that there are no gaps of nonexistent memory locations.
- Consider an example to understand the effect of memory interleaving

  Cache blocks are **of 8 words each**.

  On a read miss in the cache the DRAM is accessed. Now consider it **takes 1 clock cycle** to send the address to the main memory. The first word access in the **main memory takes 8 cycles** and subsequent accesses take 4 clock cycles per word. Calculate access time of one block (8 words from the main memory).

  Case 1: No memory interleaving that is only one memory bank/module

  Access time = 1 + 8 + (7x 4) + 1=38 cycles.

  Case 2: Memory interleaving having 4 memory banks

Once the address arrives to the main memory, in the first 8 clock cycles the 4 memory banks have 4 words ready to be sent. They are sent one word at a time in the next 4 clock cycles. During this time the next 4 words are being accessed simultaneously in each block and sending them takes another 4 cycles (1 cycle/word)

Access time= 1+8+4+4= 17 cycles.

Thus block transfer time is reduced by more than a factor of 2.

Caches:

- Caches are high speed memories realized using SRAM technology that stores a small subset of the data in the main memory and that the CPU can access directly and with minimal delay. Data is transferred in and out of the cache as and when required by the CPU.
- Caches nowadays are included on the processor chip itself. This is mainly because when data is transferred between different chips (i.e. if cache is present as a separate chip), considerable delays are introduced.
- However since processor chip size can't be increased beyond a certain size cache size on processor chips are usually very limited. All high performance processors have some form of (no matter how small) on chip cache memory. This cache can again be divided into separate instruction and data caches. This is called a Split cache. Having a combined cache offers greater hit rate as it offers flexibility in mapping new information in the cache. Split caches on the other hand make it possible to simultaneously access both the caches. This leads to increased performance but needs more complicated circuitry for the parallel access.
- High Performance Processors have also included multiple levels of cache. L1 cache is present on chip and L2 is implemented using SRAM technology and is external to the processor. However, L2 cache in a much smaller size can be included on chip as well thus giving multi-level on chip cache.
- If two levels of cache are used and as L1 is more close to the processor it is important to design the L1 cache in such way that promotes faster access by the

processor. This is because its access time has a large effect on the clock rate of the processor. A cache even if it is on chip cannot be accessed at the same speed as the CPU registers. The cache is much bigger and complex.

- Accesses to cache can be speeded up by accessing multiple words at the same time and then transferring it one by one to the processor for execution. This technique is used in many processors.
- The second level and subsequent levels of cache are now designed to be much slower than the L1 cache but have to be big enough to have a high hit rate.

11. Write short note on L1, L2 and L3 cache memory.

1.Cache is a small but fast memory device that the CPU can access at relatively faster speeds and that holds a subset of the data in the main memory. They store information the CPU is most likely to need next. If the data item is present in the cache, it is termed as a cache hit otherwise it is a cache miss. A typical organization is shown in Figure 10.

2.The same reason we have a memory hierarchy prompts us to have multi-level caches as well.A cache miss, on the other hand, means the CPU has to go scampering off to find the data elsewhere. This is where the L2 cache comes into play — while it's slower, it's also much larger. If data can't be found in the L2 cache, the CPU continues down the chain to L3 and so on.

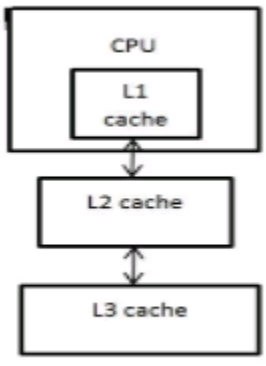


Figure 10

3.L1 caches are designed to be the fastest as they are closest to the CPU and thus it will be accessed typically by the CPU and the access time of L1 cache has a major effect on the clock rate of the CPU.L1 caches are normally much smaller than the other levels of cache

but is much bigger than the CPU's registers.L1 cache is normally on chip with processor as shown in the diagram.

4.However recent processors are also known to have multiple levels of cache on the processor chip. Since the processor chip needs to be of a certain size, this highly limits the size of on chip cache. Thus external cache levels are also fairly common.L1 is the smallest in size and gives fastest access.

5.L2 on the other hand is relatively slower but is bigger in size giving higher hit rates. L3 is slower as far as the access time is considered (not as slow as the main memory) and even bigger when the size is considered. This continues for all the cache levels.

6.Some processors use an inclusive cache design (meaning data stored in the L1 cache is also duplicated in the L2 cache) while others have an exclusive cache design (meaning the two caches never share data).

7.The access time of the caches also depends on whether the caches are on chip with the processor or are external to the processor.

8.If u consider a three level cache memory having L1, L2 and L3, the average access time (t) is given by

t=h1C1+(1−h1)h2C2+(1−h1)(1−h2)C3+(1−h1)(1−h2)(1−h3)Mt=h1C1+(1−h1)h2C2+(1−h1)(1−h2)C3+(1−h1)(1−h2)(1−h3)M
h1=Hit rate in L1

h2 = Hit rate in L2

h3 = Hit rate in L3

C1= Access Time of L1

C2= Access Time of L2

C3= Access Time of L3

M=Access Time of Main memory

The number of misses in the L2 and L3 cache given be the terms (1−h1)(1−h2) and (1−h1)(1−h2)(1−h3)(1−h1)(1−h2) and (1−h1)(1−h2)(1−h3)sh ould be as low as possible. This is the reason why L2 and L3 are larger to get higher hit rates and lower miss rates, resulting in very few main memory accesses.

10. What are the different types of buses in computer architecture?

A *bus is defined as a system that transfers data between hardware components of a computer or between two separate computers*.

System bus is a single bus that helps all major components of a computer to communicate with each other. **It is made up of an address bus, data bus and a control bus**.

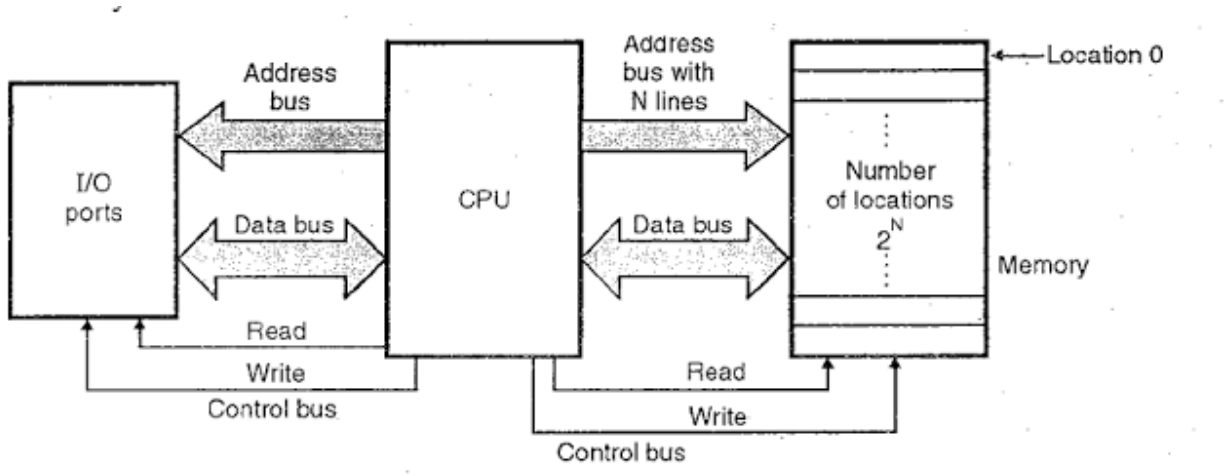


Fig. 1.2.3 : The three types of buses and their utility

The **data bus** carries the **data to be stored**, while **address bus** carries the **location** to where it should be stored.

***1) Address Bus* : Address bus is a part of the computer system bus that is dedicated for specifying a physical address**.

--> When the computer processor needs to read or write from or to the memory, it uses the address bus to specify the physical address of the individual memory block it needs to access (the actual data is sent along the data bus).

--> More correctly, when the processor wants to write some data to the memory, it will assert the write signal, set the write address on the address bus and put the data on to the data bus.

Similarly, when the processor wants to read some data residing in the memory, it will assert the read signal and set the read address on the address bus. After receiving this signal, the memory controller will get the data from the specific memory block (after checking the address bus to get the read address) and then it will place the data of the memory block on to the data bus.

Good to know : The size of the memory that can be addressed by the system determines the width of the data bus and vice versa. For example, if the width of the address bus is 32 bits, the system can address 2^32 memory blocks (that is equal to 4GB memory space, given that one block holds 1 byte of data).

***2) Data Bus* : A data bus simply carries data**.

Typically, the same data bus is used for both read/write operations.

--> When it is a write operation, the processor will put the data (to be written) on to the data bus.

--> When it is the read operation, the memory controller will get the data from the specific memory block and put it in to the data bus.

Good to know :

--> The data bus consist of 8, 16, or 32 parallel lines.

--> The data bus is bidirectional bus, means the data can be transferred from CPU to main memory and vice versa.

--> The number of data lines used in the data bus is equal to the size of data word being written or read.

--> The data bus also connects the I/O ports and CPU. So, the CPU can write data to or read it from the memory or I/O ports.

What is the difference between Address Bus and Data Bus?

-> Data bus is bidirectional, while address bus is unidirectional.

--> That means data travels in both directions but the addresses will travel in only one direction. The reason for this is that unlike the data, the *address is always specified by the processor*.

The *width of the data bus is determined by the size of the individual memory block*, while the *width of the address bus is determined by the size of the memory that should be addressed by the system*.

Computers have two major types of buses:
1. System bus:- This is the bus that connects the CPU to main memory on the motherboard. The system bus is also called the front-side bus, memory bus, local bus, or host bus.
2. A number of I/O Buses, (I/O is an acronym for input / output), connecting various peripheral devices to the CPU. These devices connect to the system bus via a 'bridge' implemented in the processors chipset. Other names for the I/O bus include "expansion bus", "external bus" or "host bus".

Expansion Bus Types

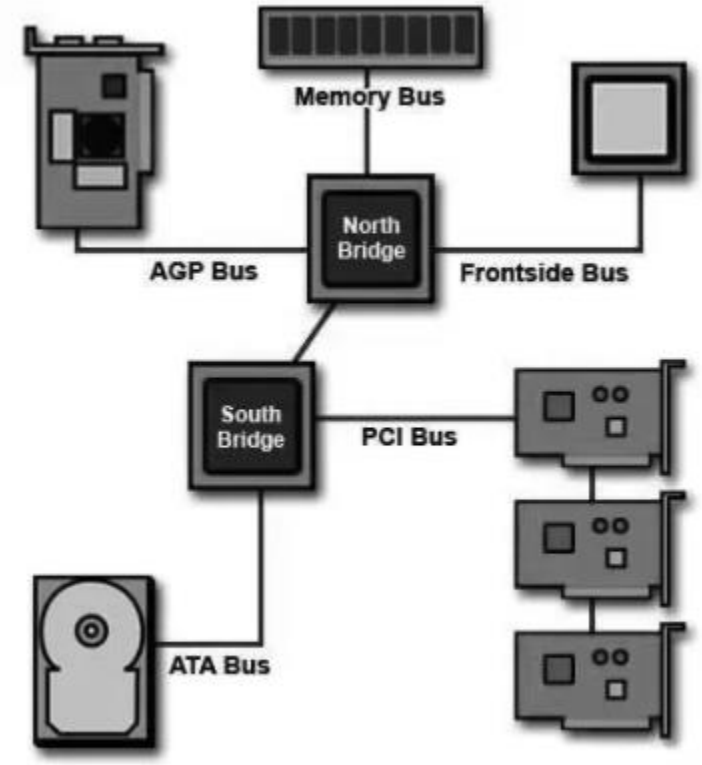
These are some of the common expansion bus types that have ever been used in computers:

- **ISA** - Industry Standard Architecture
- **EISA** - Extended Industry Standard Architecture
- **MCA** - Micro Channel Architecture
- **VESA** - Video Electronics Standards Association
- **PCI** - Peripheral Component Interconnect
- **PCMCIA** - Personal Computer Memory Card Industry Association (Also called PC bus)
- **AGP** - Accelerated Graphics Port
- **SCSI** - Small Computer Systems Interface

An expansion bus provides an input/output pathway for transferring information between internal hardware, such as RAM or the CPU, and expansion devices such as a graphics card

or sound card.

In other word, Expansion bus or External bus allows the CPU to talk to the other devices in the computer and vice versa. It is called External bus because it's external to the CPU.



There are several different types of expansion buses with diverse features, such as:

- Industry Standard Architecture (ISA)
- Accelerated Graphics Port (AGP)
- Peripheral Component Interconnect (PCI)
- PCI Express (PCI-X)

Expansion Bus Chart:

Type of Bus	Bits Wide	Clock Speed	Transfer Speed
ISA	8 bit	4.77 MHz	2.38MB/s
ISA	16 bit	8.33 MHz	8MB/s
PCI (Client)	32 bit	33MHz	133MB/s
PCI (Server)	64 bit	66MHz	266MB/s
AGP 1x	32 bit	66MHz	266MB/s
AGP 2x	32 bit	66MHz	533MB/s
AGP 4x	32 bit	66MHz	1,066MB/s
AGP 8x	32 bit	66MHz	2,133MB/s
AGP 8x (high-end)	64 bit	66MHz	4,266MB/s

11. What is bus arbitration? Explain any two techniques of bus arbitration.

- Bus arbitration is the process by which the next device to become the bus master is selected and bus mastership is transferred to it. The selection of bus master is usually done on the priority basis.
- There are two approaches to bus arbitration: Centralized and distributed.

1. Centralized Arbitration

- In centralized bus arbitration, a single bus arbiter performs the required arbitration. The bus arbiter may be the processor or a separate controller connected to the bus.
- There are three different arbitration schemes that use the centralized bus arbitration approach. There schemes are:

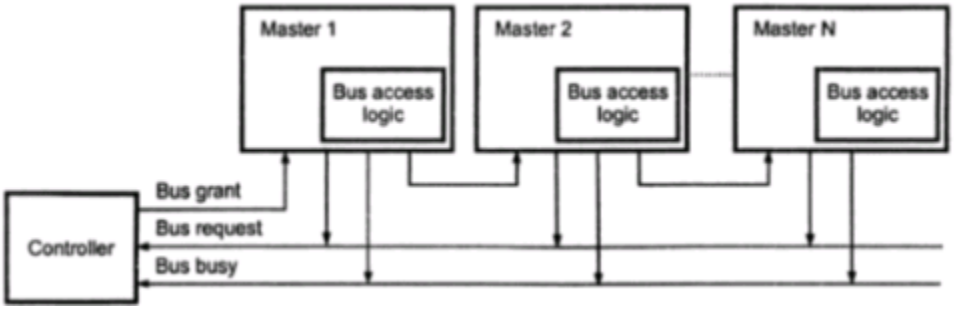
  a. Daisy chaining

  b. Polling method

  c. Independent request

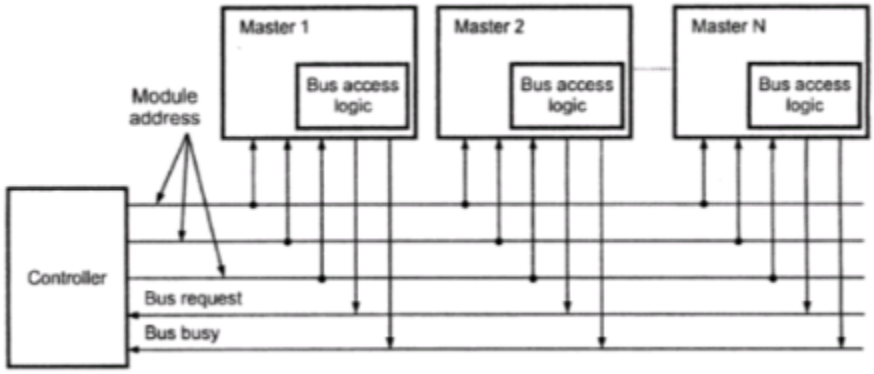
a) Daisy chaining

- The system connections for Daisy chaining method are shown in fig below.



- It is simple and cheaper method. All masters make use of the same line for bus request.
- In response to the bus request the controller sends a bus grant if the bus is free.
- The bus grant signal serially propagates through each master until it encounters the first one that is requesting access to the bus. This master blocks the propagation of the bus grant signal, activities the busy line and gains control of the bus.
- Therefore any other requesting module will not receive the grant signal and hence cannot get the bus access.

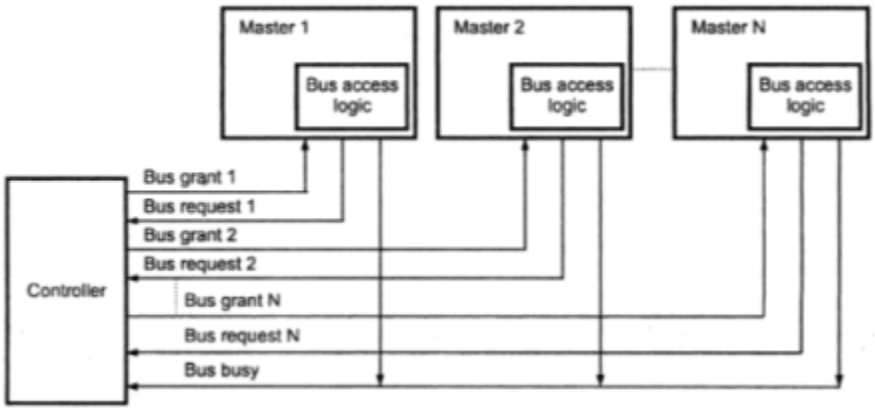
b) Polling method



- The system connections for polling method are shown in figure above.
- In this the controller is used to generate the addresses for the master. Number of address line required depends on the number of master connected in the system.

- For example, if there are 8 masters connected in the system, at least three address lines are required.
- In response to the bus request controller generates a sequence of master address. When the requesting master recognizes its address, it activated the busy line ad begins to use the bus.

c) Independent request



- The figure below shows the system connections for the independent request scheme.
- In this scheme each master has a separate pair of bus request and bus grant lines and each pair has a priority assigned to it.
- The built in priority decoder within the controller selects the highest priority request and asserts the corresponding bus grant signal.

2. Distributed Arbitration

- In distributed arbitration, all devices participate in the selection of the next bus master.
- In this scheme each device provided with arbitration hardware and is assigned a 4-bit identification number.
- The number of devices connected on the bus when one or more devices request for the control of bus, they assert the start-arbitration signal and place their 4-bit ID numbers on arbitration lines, ARB0 through ARB3.
- These four arbitration lines are all open-collector. Therefore, more than one device can place their 4-bit ID number to indicate that they need to control of bus. When two or more devices place their ID number on bus lines then it is necessary to identify the highest ID

number on bus lines then it is necessary to identify the highest ID number from the status of bus line.

- The decentralized arbitration offers high reliability because operation of the bus is not dependent on any single device.

Bus arbitration schemes usually try to balance:

i. Fixed priority: The highest priority device should be serviced first.\
ii. Rotation priority: Priority is not fixed. There are several ways of changing priority eg: Increase the priority as a function of waiting time or lowest priority for the master that just received the bus etc..
iii. Fairness: Even the lowest priority device should never be completely locked out from the bus. It need not use priorities. Fairness can be defined in several ways. One of techniques is window based satisfaction technique.

- Bus release policy can be preemptive or non-preemptive.

12. Describe the following elements of Bus Design: A) Bus Types B) Bus Timing C) data transfer type

BUS TYPES:

- Bus channels can be separated into two general types, namely a dedicated and multiplexed. A bus line dedicated permanently assigned a function or a subset of physical computer components.
- As an example of dedication to the function is the use of separate dedicated address and data lines, which is a common thing for the bus.
- However, this is not important. For example, the address and data information can be transmitted through the same number of channels using control channel address is invalid. In the early transfer of data, address bus and address placed on a valid control activated. At this time, each module has a specific time period to copy the address and determine whether the address is a module located. Then address removed from the bus and the bus connection is used for reading or writing data transfer next. Methods use the same line for various purposes is known as time multiplexing.

- The advantage is time multiplexing requires fewer channels, which saves space and cost. The disadvantage is the need for a more complex circuit within each module. There's also a fairly large decrease in performance due to certain events that use the channels together cannot function in parallel.
- Physical dedication associated with the use of multiple buses, each bus it is connected with only a subset of the modules. A common example is the use of bus I / O to interconnect all I / O module, then this bus is connected to the main bus through a kind of adapter module I / O. The main advantage of physical dedication is a high throughput; because of the traffic congestion is only small data. The disadvantage is the increased size and cost of the system.

B) bus timing

Can be synchronous or asynchronous.

A **synchronous** bus is **clocked**.

- One of the lines in the bus is a clock that serves as the clock for all the devices on the bus.
- All the bus actions are done on fixed clock cycles. For example, 4 cycles after receiving a request, the memory delivers the first word.
- This can be handled by a simple finite state machine (FSM). Basically, once the request is seen everything works one clock at a time. Because the protocol is so simple, it requires few gates and is very fast.
- Processor to memory buses are now normally synchronous.
    - The number of devices on the bus are small.
    - The bus is short.
    - The devices (i.e. processor and memory) are prepared to run at the same speed.
- Two problems with synchronous buses.
    1. All the devices must run at the same speed.
2. The bus must be short due to *clock skew*.

An **asynchronous** bus is **not** clocked.

- Since the bus is not clocked devices of varying speeds can be on the same bus.
- There is no problem with clock skew (since there is no clock).
- But the bus must now contain control lines to coordinate transmission.
- Common is a handshaking protocol.

13. Explain various RAID levels.

- **RAID** (Redundant Array of Independent Disks) is a data storage virtualization technology that combines multiple physical disk drive components into a single logical unit for the purposes of data redundancy, performance improvement, or both.
- Data is distributed across the drives in one of several ways, referred to as RAID levels, depending on the required level of redundancy and performance.
- The different schemas, or data distribution layouts, are named by the word RAID followed by a number, for example RAID 0 or RAID 1.
- Each schema, or a RAID level, provides a different balance among the key goals: reliability, availability, performance, and capacity.
- RAID levels greater than RAID 0 provide protection against unrecoverable sector read errors, as well as against failures of whole physical drives.
- Two techniques commonly used are mirroring and striping:
    - Mirroring refers to copying of data to multiple hard disks.
    - Striping refers to splitting of data across multiple hard disks

RAID 0:

- In a RAID 0 configuration, data is **striped** across the HDDs in a RAID set. It utilizes the full storage capacity by distributing strips of data over multiple HDDs in a RAID set.
- To read data, all the strips are put back together by the controller.

- The stripe size is specified at a host level for software RAID and is vendor specific for hardware RAID.
- Figure below shows RAID 0 on a storage array in which data is striped across 5 disks. When the number of drives in the array increases, performance improves because more data can be read or written simultaneously.
- RAID 0 is used in applications that need high I/O throughput. However, if these applications require high availability, RAID 0 does not provide data protection and availability in the event of drive failures.



Fig: RAID 0

RAID 1:

- In a RAID 1 configuration, data is **mirrored** to improve fault tolerance. A RAID 1 group consists of at least two HDDs.
- As explained in mirroring, every write is written to both disks, which is transparent to the host in a hardware RAID implementation.
- In the event of disk failure, the impact on data recovery is the least among all RAID implementations. This is because the RAID controller uses the mirror drive for data

recovery and continuous operation. RAID 1 is suitable for applications that require high availability.

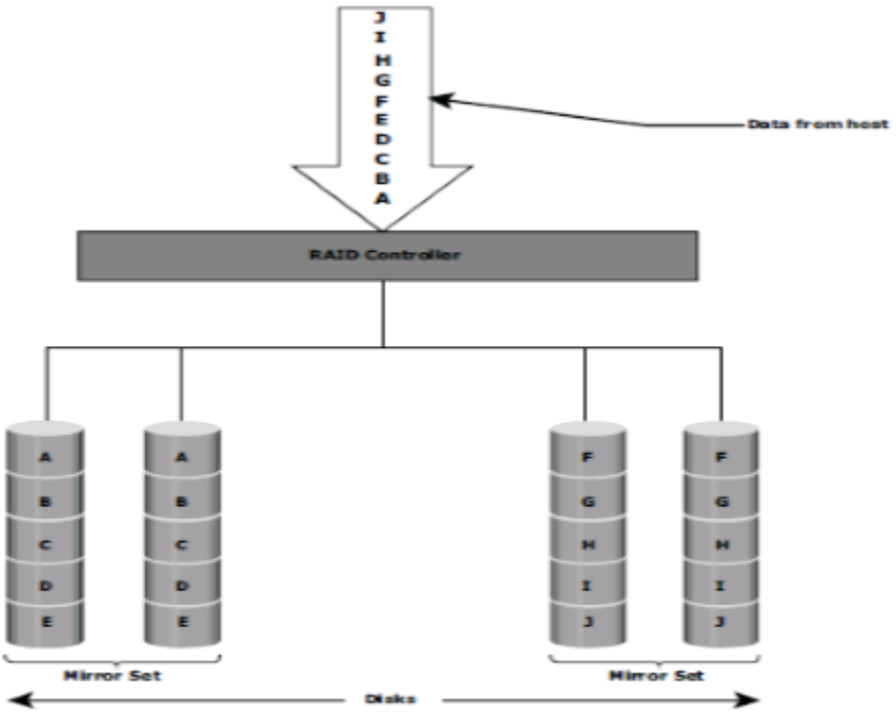


Fig: RAID 1

RAID 3:

- RAID 3 **stripes data** for high performance and **uses parity** for improved fault tolerance. Parity information is stored on a dedicated drive so that data can be reconstructed if a drive fails.
- Figure illustrates the RAID 3 implementation. RAID 3 is used in applications that involve large sequential data access, such as video streaming.

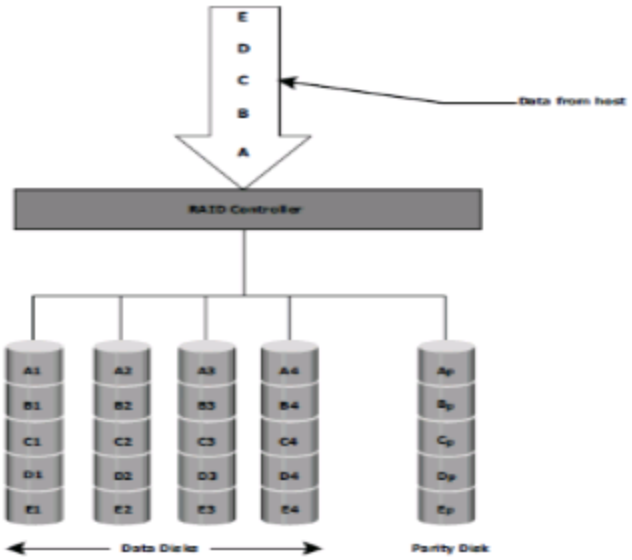


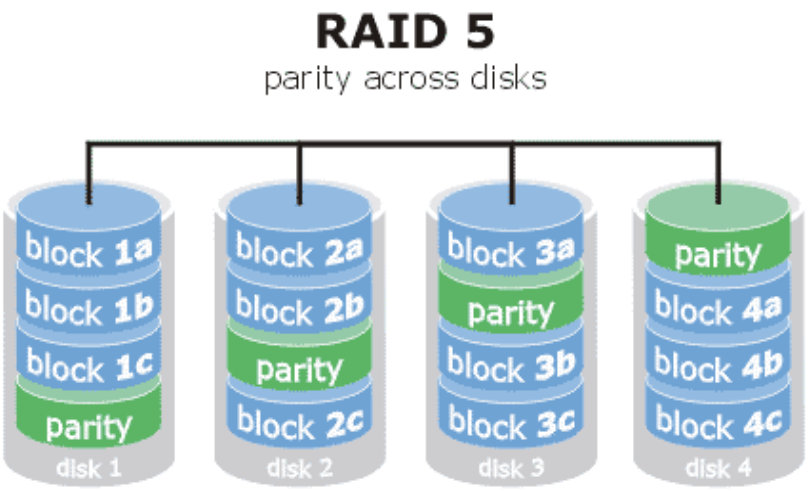
Fig: RAID 3

RAID 4:

- Similar to RAID 3, RAID 4 **stripes data** for high performance and uses parity for improved fault tolerance.
- Data is striped across all disks except the parity disk in the array. Parity information is stored on a dedicated disk so that the data can be rebuilt if a drive fails. Striping is done at the **block level** unlike in RAID 3 where it is done in byte level.

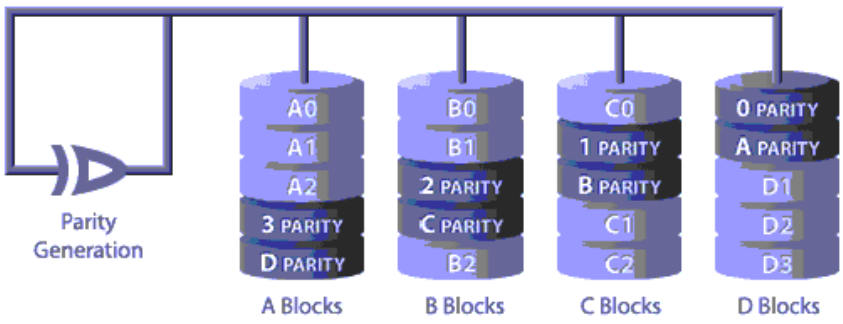
RAID 5:

- RAID 5 is a very versatile RAID implementation. It is similar to RAID 4 because it uses striping and the drives (strips) are independently accessible.
- The difference between RAID 4 and RAID 5 is the parity location. In RAID 4, parity is written to a dedicated drive, creating a write bottleneck for the parity disk. In RAID 5, parity is distributed across all disks.
- The distribution of parity in RAID 5 overcomes the write bottleneck.
- Figure illustrates the RAID 5 implementation. RAID 5 is preferred for messaging, data mining, and medium-performance media serving, and relational database management system (RDBMS) implementations in which database administrators (DBAs) optimize data access.



RAID 6:

- RAID 6 works the same way as RAID 5 except that RAID 6 includes a second parity element to enable survival in the event of the failure of two disks in a RAID group.
- Therefore, a RAID 6 implementation requires at least four disks.
- RAID 6 distributes the parity across all the disks. The write penalty in RAID 6 is more than that in RAID 5; therefore, RAID 5 writes perform better than RAID 6.
- The rebuild operation in RAID 6 may take longer than that in RAID 5 due to the presence of two parity sets.



RAID 2 is similar to RAID 5, but instead of disk striping using parity, **striping occurs at the bit-level**. RAID 2 is seldom deployed because costs to implement are usually

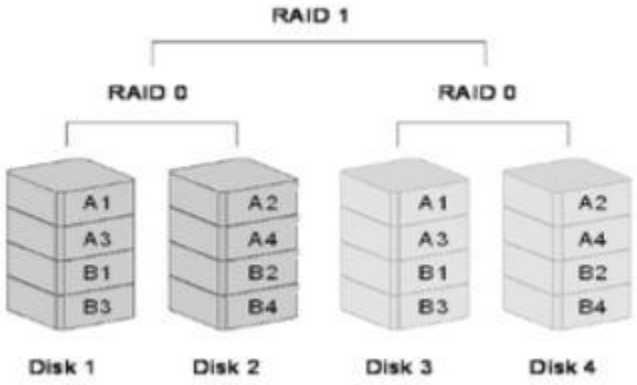
prohibitive (a typical setup requires 10 disks) and gives poor performance with some disk I/O operations.

14. **Explain nested RAID with its application**

- **Nested RAID levels, also known as hybrid RAID,** combine two or more of the standard RAID levels to gain performance, additional redundancy or both, as a result of combining properties of different standard RAID layouts.
- Data centers require data redundancy and performance from their RAID arrays. RAID 0+1 and RAID 1+0 combine the performance benefits of RAID 0 with the redundancy benefits of RAID 1. They use striping and mirroring techniques and combine their benefits. These types of RAID require an even number of disks, the minimum being four. RAID 1+0 is also known as RAID 10 (Ten) or RAID 1/0. Similarly, RAID 0+1 is also known as RAID 01 or RAID 0/1. RAID 1+0 performs well for workloads that use small, random, write-intensive I/O.
- **RAID 0+1**

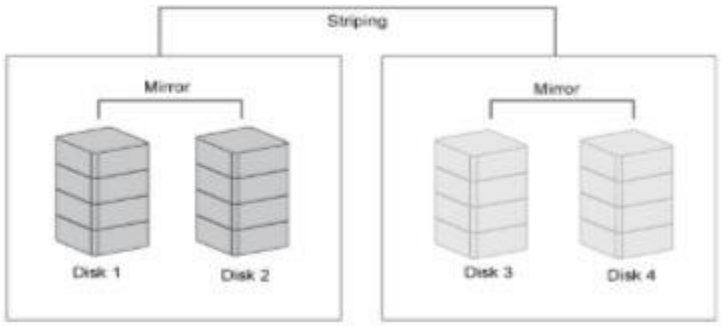
  A RAID 0+1 (also called RAID 01), is a RAID level used for both replicating and sharing data among disks. The minimum number of disks required to implement this level of RAID is 3 but it is more common to use a minimum of 4 disks. The difference between RAID 0+1 and RAID 1+0 is the location of each RAID system — RAID 0+1 is a **mirror of stripes** although some manufacturers (e.g. Digital/Compaq/HP) use RAID 0+1 to describe striped mirrors, consequently this usage is now deprecated so that RAID 0+1 and RAID1+0 are replaced by RAID10 whose definition correctly describes the correct and safe layout ie

  Stripped mirror.



- **RAID 1+0**

  A RAID 1+0, sometimes called RAID 1&0 or RAID 10, is similar to a RAID 0+1 with exception that the RAID levels used are reversed RAID 10. RAID 10, as recognized by the storage industry association and as generally implemented by RAID controllers, is a RAID 0 array of mirrors, which may be two-way or three way-mirrors, and requires a minimum of four drives. More than four disks are possible in RAID 10 and these larger arrays are common in professional applications.



Applications that benefit from RAID 1+0 include the following:

- High transaction rate Online Transaction Processing (OLTP)
- Large messaging installations
- Database applications that require high I/O rate, random access, and high availability
- In most cases RAID 1+0 provides better throughput and latency than all other RAID levels except RAID 0 which wins in throughput. Thus, it is the preferable RAID level for I/O-

intensive applications such as database, email, and web servers, as well as for any other use requiring high disk performance.

- **RAID level descriptions:**

RAID	Description
0	consists of striping, without mirroring or parity
1	consists of data mirroring, without parity or striping
2	consists of bit-level striping with dedicated Hamming-code parity
3	consists of byte-level striping with dedicated parity
4	consists of block-level striping with dedicated parity
5	consists of block-level striping with distributed parity
6	consists of block-level striping with double distributed parity
0+1	creates a second striped set to mirror a primary striped set
1+0	creates a striped set from a series of mirrored drives

15. Explain different modes of transfer for data.

There are three modes of transfer for data, commands and status as follows:

1. Programmed IO
2. Interrupt driven IO
3. Direct memory access

Programmed I/O is implicated to data transfers that are initiated by a CPU.

With programmed I/O, data are exchanged between the processor and the I/O module. The processor executes a program that gives it direct control of the I/O operation, including sensing device status, sending a read or write command, and transferring the data. When the processor issues a command to the I/O module, it must wait until the I/O operation is complete.

- If the processor is faster than the I/O module, this is wasteful of processor time. With interrupt-driven I/O, the processor issues I/O command, continues to execute other instructions, and is interrupted by the I/O module when the latter has completed its work.
- With both programmed and interrupt I/O, the processor is responsible for extracting data from main memory for output and storing data in main memory for input. The alternative is known as direct memory access. In this mode, the I/O module and main memory exchange data directly, without processor involvement.
- With programmed I/O, the I/O module will perform the requested action and then set the appropriate bits in the I/O status register. The I/O module takes no further action to alert the processor.

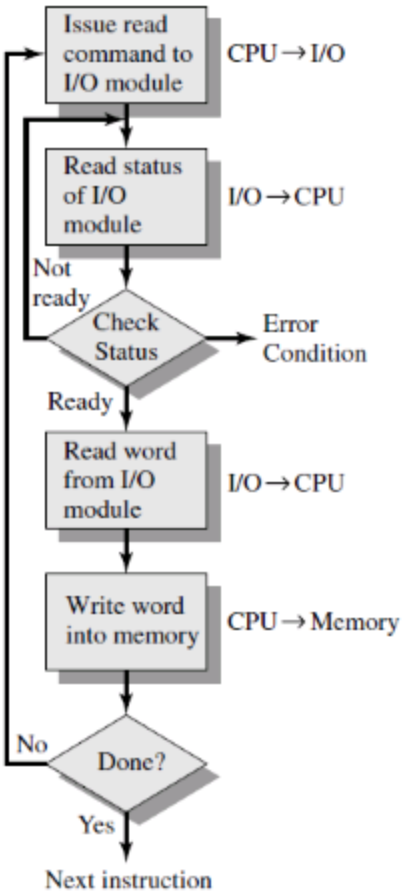


Figure 6

- As shown in Figure 6, when the processor is executing a program and encounters an instruction relating to I/O, it executes that instruction by issuing a command to the appropriate I/O module. In particular, it does not interrupt the processor. Thus, it is the responsibility of the processor periodically to **check the status** of the I/O module. Then if the device is ready for the transfer (read/write). The processor transfers the data to or from the I/O device as required. As the CPU is faster than the I/O module, the problem with programmed I/O is that the CPU has to wait a long time for the I/O module of concern to be ready for either reception or transmission of data.

  i. The CPU, while waiting, must repeatedly check the status of the I/O module, and this process is known as Polling.

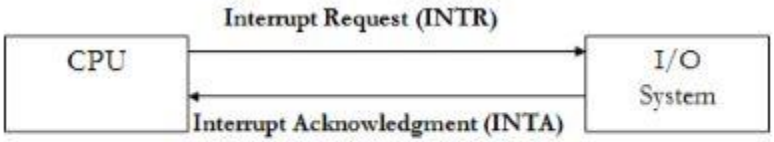
  ii. Therefore, the level of the performance of the entire system is severely degraded.

Interrupt driven technique is used to overcome the limitation of programmed I/O.

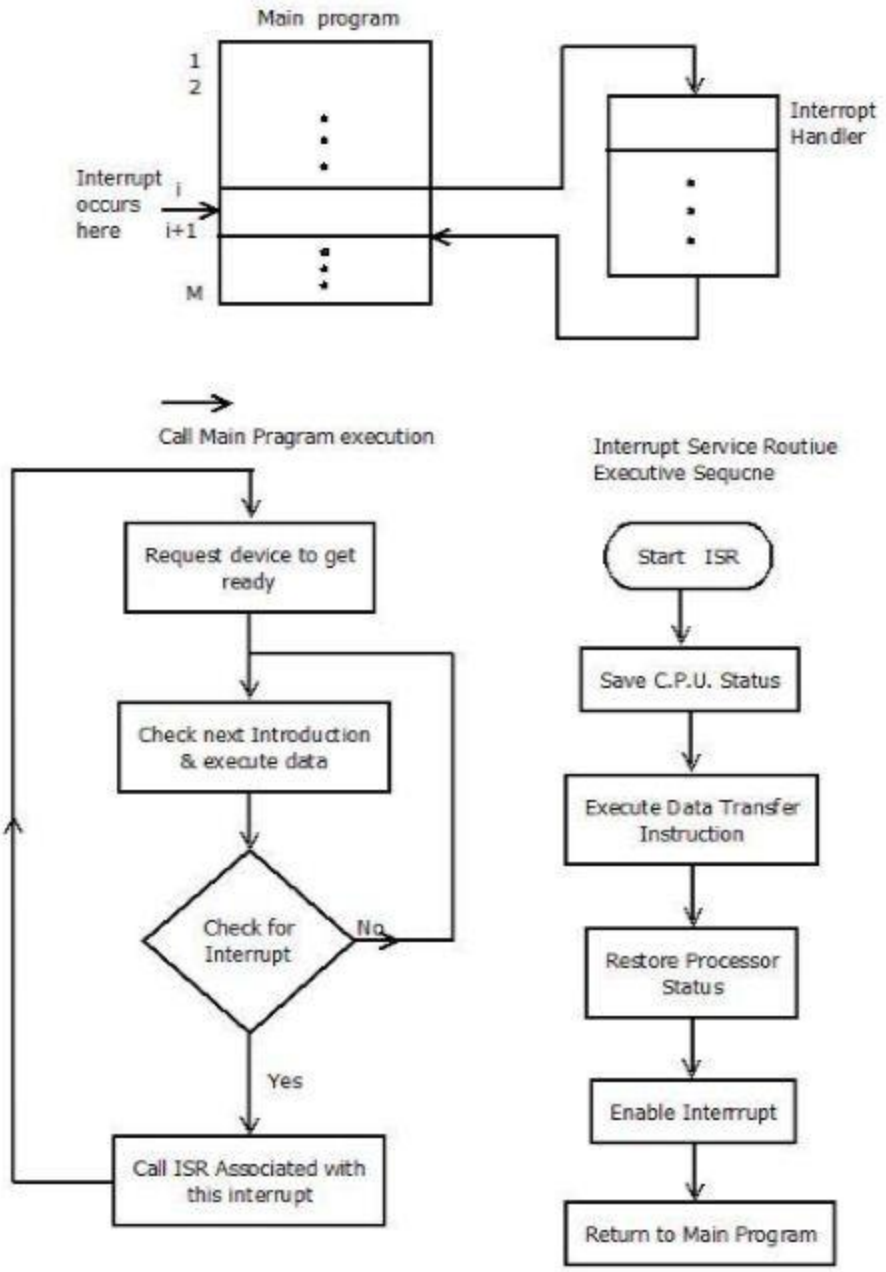
- In interrupt driven I/O, instead of making the processor to verify the status of I/O module. It is the responsibility of I/O module to intimate the processor by interrupt signal.
- CPU responds to interrupt signals and stores the return address from the program counter (PC) into the memory stack and then the control branches to a interrupt service routine (ISR).
- That is whenever I/O device is ready for the data transfer, it will interrupt the CPU. In the ISR, the CPU will perform the data transfer.
- After completion of executing interrupt routine CPU returns to previous program and continue what it was doing before.

Interrupt:

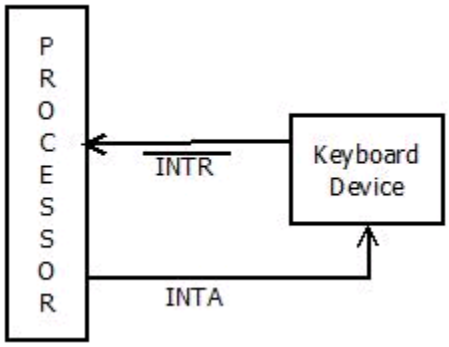
- An interrupt or exception causes CPU to transfer the control temporarily from its current program to another program i.e. interrupt handler.
- Block Diagram for Interrupt Driven I/O



Transfer control from main program to Interrupt Handler.



- The interrupt driven I/O access can be explained with the example of keyboard.

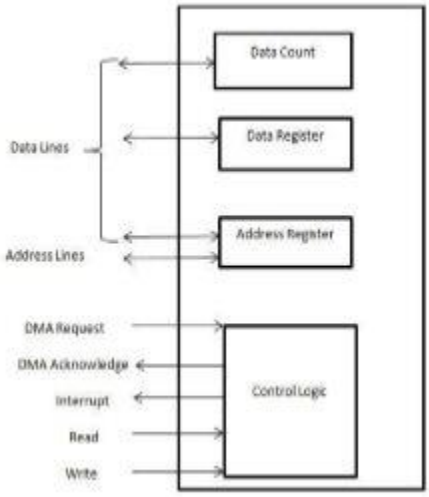


- Instead of CPU checking, when the data is available on the keyboard, the keyboard should interrupt the CPU when a key is pressed. Thus time will not be wasted in repeatedly checking the keyboard when the user is not typing at all.
- The I/O device that request an interrupt by sending INTR' signal to the CPU.
- Whenever CPU receives INTR' signal, it completes the execution of the current instruction and then executes the ISR. When CPU is ready then it send acknowledge signal through INTA line.
- ISR gets executed and CPU returns to the main program.
- I/O devices turns off the INTR' signal after execution.

Direct Memory Access (DMA) allows devices to transfer data without subjecting the processor a heavy overhead. Otherwise, the processor would have to copy each piece of data from the source to the destination. This is typically slower than copying normal blocks of memory since access to I/O devices over a peripheral bus is generally slower than normal system RAM. During this time the processor would be unavailable for any other tasks involving processor bus access. But it can continue to work on any work which does not require bus access.

1. When large volumes of data are to be moved, a more efficient technique is required: direct memory access (DMA).
2. DMA involves an additional module on the system bus. This module is called the DMA module and it is shown in Figure 4. The DMA module is capable of mimicking the processor and indeed, of taking over control of the system from the processor. It needs to do this to transfer data to and from memory over the system bus.

3. The DMA module must use the bus only when the processor does not need it, or it must force the processor to suspend operation temporarily.



1. It is also referred to as cycle stealing, because the DMA module in effect steals a bus cycle. When the processor wishes to read or write a block of data, it issues a command to the DMA module, by sending to the DMA module the following information:

   i. Whether a read or write is requested, using the read or write control line between the processor and the DMA module

   ii. The address of the I/O device involved, communicated on the data lines

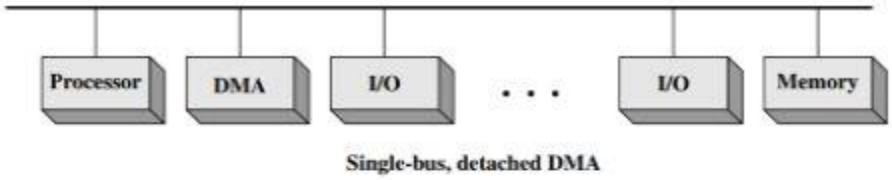
   iii. The starting location in memory to read from or write to, communicated on the data lines and stored by the DMA module in its address register

   iv. The number of words to be read or written, again communicated via the data lines and stored in the data count register

2. The Control Logic in the DMA module is responsible for the generation of control signals.
3. The processor then continues with other work. It has delegated this I/O operation to the DMA module. The DMA module transfers the entire block of data, one word at a time, directly to or from memory, without going through the processor. When the transfer is complete, the DMA module sends an interrupt signal to the processor.
4. Thus, the processor is involved only at the beginning and end of the transfer. In the instruction cycle the processor may be suspended. In each case, the processor is suspended

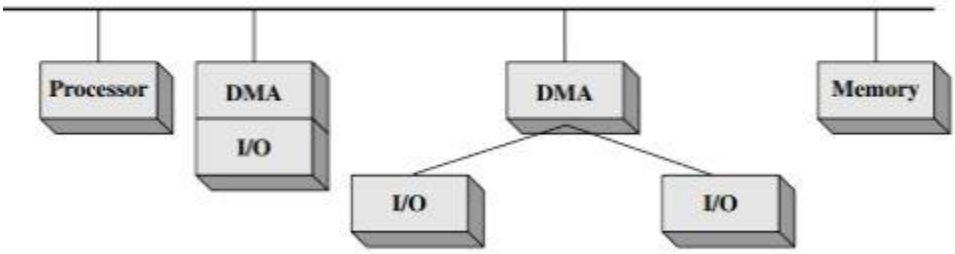
just before it needs to use the bus. The DMA module then transfers one word and returns control to the processor.

5. Note that this is not an interrupt; the processor does not save a context and do something else. Rather, the processor pauses for one bus cycle. The overall effect is to cause the processor to execute more slowly. Nevertheless, for a multiple-word I/O transfer, DMA is far more efficient than interrupt-driven or programmed I/O.



Single-bus, detached DMA

Figure 5(a)



Single-bus, integrated DMA-I/O

Figure 5(b)

1. The configuration explained above is shown in Figure 5(a), it is inexpensive but inefficient. As with processor-controlled programmed I/O, each transfer of a word consumes two bus cycles. The number of required bus cycles can be cut substantially by integrating the DMA and I/O functions, as Figure 5(b) shows. There is a path between the DMA module and one or more I/O modules that does not include the system bus.

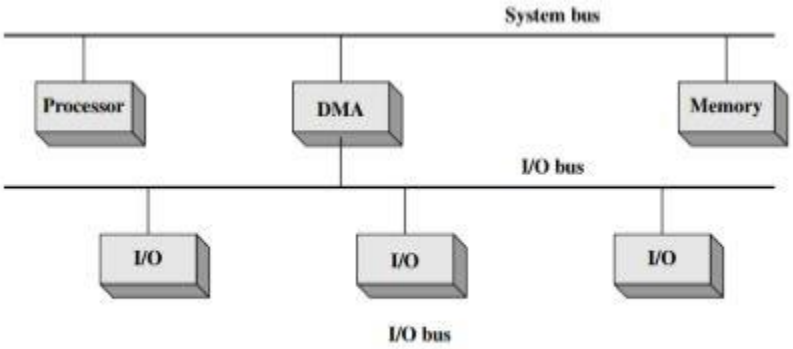


Figure 5(c)

1. The DMA logic may actually be a part of an I/O module, or it may be a separate module that controls one or more I/O modules. This concept can be taken one step further by connecting I/O modules to the DMA module using an I/O bus; this is shown in Figure 5(c). This reduces the number of I/O interfaces in the DMA module to one and provides for an easily expandable configuration.
2. In both cases Figure 5(b) and Figure5(c), the system bus that the DMA module shares with the processor and memory is used by the DMA module only to exchange data with memory. The exchange of data between the DMA and I/O modules takes place off the system bus.

The DMA Controller has following modes of data transfer. They are:

1. Cycle Steal:

A read or write signal is generated by the DMAC, and the I/O device either generates or latches the data. The DMAC effectively steals cycles from the processor in order to transfer the byte, so single byte transfer is also known as cycle stealing.

i. Requests by DMA devices for using the bus are always given higher priority than processor requests.

ii. Among different DMA devices, top priority is given to high-speed peripherals such as disks, high-speed network interface, and graphics display device).

iii. Since the processor initiates most memory access cycles, it is often stated that DMA steals memory cycles from the processor (cycle stealing) for its purpose.

iv. If DMA controller is given exclusive access to the main memory to transfer a block of data without interruption, this is called block or burst mode.

2. Burst Transfer:

i. To achieve block transfers, some DMAC's incorporate an automatic sequencing of the value presented on the address bus. A register is used as a byte count, being decremented for each byte transfer, and upon the byte count reaching zero, the DMAC will release the bus. When the DMAC operates in burst mode, the CPU is halted for the duration of the data transfer.

ii. In burst mode, the processor is stopped completely until the DMA transfer is completed. Although the processor has no control over its system during such a delay, this mode appears to be more appropriate when predictability is the main goal. The main disadvantage being that the CPU is halted for the time when the DMA is in control of the bus.

3. Hidden mode/ Transparent mode:

i. It is possible to perform hidden DMA, which is transparent to the normal operation of the CPU. In other words, the bus is grabbed by the DMAC when the processor is not using it. The DMAC monitors the execution of the processor, and when it recognizes the processor executing an instruction which has sufficient empty clock cycles to perform a byte transfer; it waits till the processor is decoding the op code and then grabs the bus during this time.

ii. The processor is not slowed down, but continues processing normally. Naturally, the data transfer by the DMAC must be completed before the processor starts.

16. Compare SRAM and DRAM.

- Following table lists the major types of semiconductor memory.
- The most common is referred to as random-access memory (RAM).
- This is, of course, a misuse of the term, because all of the types listed in the table are random access.
- One distinguishing characteristic of RAM is that it is possible both to read data from the memory and to write new data into the memory easily and rapidly.
- Both the reading and writing are accomplished through the use of electrical signals.

Memory Type	Category	Erasure	Write Mechanism	Volatility
Random-access memory (RAM)	Read-write memory	Electrically, byte-level	Electrically	Volatile
Read-only memory (ROM)	Read-only memory	Not possible	Masks	Nonvolatile
Programmable ROM (PROM)			Electrically	
Erasable PROM (EPROM)	Read-mostly memory	UV light, chip-level		
Electrically Erasable PROM (EEPROM)		Electrically, byte-level		
Flash memory		Electrically, block-level		

- The other distinguishing characteristic of RAM is that it is volatile. A RAM must be provided with a constant power supply.
- If the power is interrupted, then the data are lost. Thus, RAM can be used only as temporary storage. The two traditional forms of RAM used in computers are DRAM and SRAM.

DYNAMIC RAM

- RAM technology is divided into two technologies: dynamic and static. A dynamic RAM (DRAM) is made with cells that store data as charge on capacitors.
- The presence or absence of charge in a capacitor is interpreted as a binary 1 or 0. Because capacitors have a natural tendency to discharge, dynamic RAMs require periodic charge refreshing to maintain data storage.
- The term dynamic refers to this tendency of the stored charge to leak away, even with power continuously applied.
- Figure(1) is a typical DRAM structure for an individual cell that stores 1 bit.
- The address line is activated when the bit value from this cell is to be read or written.
- The transistor acts as a switch that is closed (allowing current to flow) if a voltage is applied to the address line and open (no current flows) if no voltage is present on the address line.
- For the write operation, a voltage signal is applied to the bit line; a high voltage represents 1, and a low voltage represents 0.
- A signal is then applied to the addressline, allowing a charge to be transferred to the capacitor.
- For the read operation, when the address line is selected, the transistor turns on and the charge stored on the capacitor is fed out onto a bit line and to a sense amplifier.
- The sense amplifier compares the capacitor voltage to a reference value and determines if the cell contains a logic 1 or a logic 0. The readout from the cell discharges the capacitor, which must be restored to complete the operation.

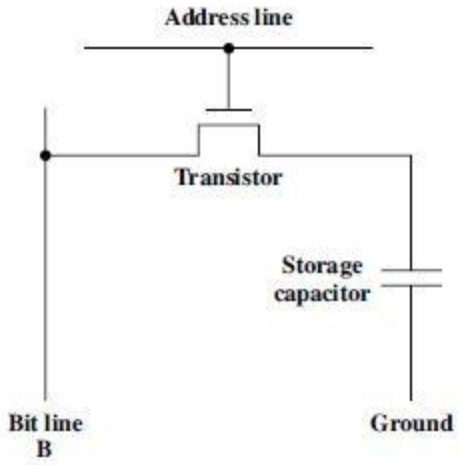


Figure 1

- Although the DRAM cell is used to store a single bit (0 or 1), it is essentially an analog device.
- The capacitor can store any charge value within a range; a threshold value determines whether the charge is interpreted as 1 or 0.

STATIC RAM

- In contrast, a static RAM (SRAM) is a digital device that uses the same logic elements used in the processor.
- In a SRAM, binary values are stored using traditional flip-flop logic-gate configurations A static RAM will hold its data as long as power is supplied to it.
- Figure (2) is a typical SRAM structure for an individual cell. Four transistors (T1, T2, T3, T4) are cross connected in an arrangement that produces a stable logic state. In logic state 1, point C1 is high and point C2 is low; in this state,T1and T4 are off and T2and T3 are on.1 In logic state 0, point C1 is low and point C2 is high; in this state,T1 and T4 are on and T2 and T3 are off.
- Both states are stable as long as the direct current (dc) voltage is applied. Unlike the DRAM, no refresh is needed to retain data.
- As in the DRAM, the SRAM address line is used to open or close a switch. The address line controls two transistors (T5 and T6).When a signal is applied to this line, the two transistors are switched on, allowing a read or write operation.
- For a write operation, the desired bit value is applied to line B, while its complement is applied to line. This forces the four transistors (T1, T2, T3, T4) into the proper state. For a read operation, the bit value is read from line B.

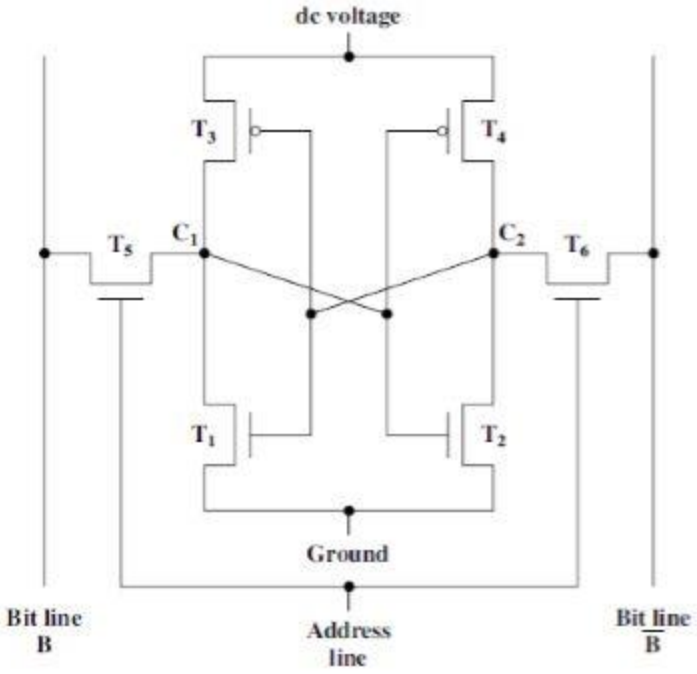
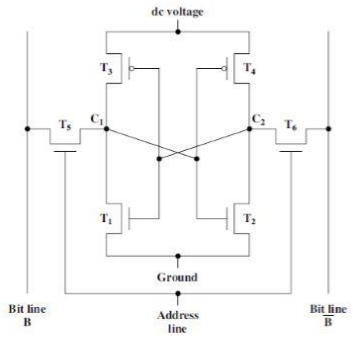
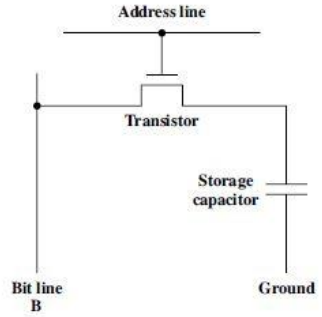


Figure 2

SRAM VERSUS DRAM

- Both static and dynamic RAMs are volatile; that is, power must be continuously supplied to the memory to preserve the bit values.
- A dynamic memory cell is simpler and smaller than a static memory cell. Thus, a DRAM is denser (smaller cells = more cells per unit area) and less expensive than a corresponding SRAM. On the other hand, a DRAM requires the supporting refresh circuitry.
- For larger memories, the fixed cost of the refresh circuitry is more than compensated for by the smaller variable cost of DRAM cells.
- Thus, DRAMs tend to be favored for large memory requirements. A final point is that SRAMs are generally somewhat faster than DRAMs.
- Because of these relative characteristics, SRAM is used for cache memory (both on and off chip), and DRAM is used for main memory.

Differences can be tabulated as follows:

	SRAM	DRAM
1	Circuit Diagram  Figure 2	Circuit Diagram  Figure 1
2	It consists of Number of flip-flops to store data.	It stores data as a change on capacitor.
3	It contains less memory cells per units area.	It contains moray memory cells per unit area.
4	It is faster.	It is slower.
5	Refreshing circuitry is not required.	Refreshing circuitry is required.
6	Cost is more.	Cost is less.
7	Size is more.	Size is less.

	SRAM	DRAM
8	Consume more power.	Consume less power.

17. Explain I/O module and its usage?

I/O modules are the critical element of the computer system after CPU and memory. All computer systems must have efficient means to receive input and deliver output. Wide variety of peripherals in different amounts of data, at different speeds and in different formats, all are slower than CPU and RAM.

Input-output module provides a method for transferring information between internal storage and external I/O devices.

Peripherals connected to a computer need special communication links for interfacing them with the central processing unit.

The requirements of the communication link are to resolve the differences that exist between the central computer and each peripheral.

The requirements are as follows:

Peripherals are electromechanical and electromagnetic devices and their manner of operation is different from the operation of the CPU and memory, which are electronic devices. Therefore, a conversion of signal values may be required.

The data transfer rate of peripherals is usually slower than the transfer rate of the CPU, and consequently, a synchronization mechanism may be needed.

Data codes and formats in peripherals differ from the word format in the CPU and memory.

The operating modes of peripherals are different from each other and each must be controlled so as not to disturb the operation of other peripherals connected to the CPU.

A typical I/O module is shown in Figure 8:

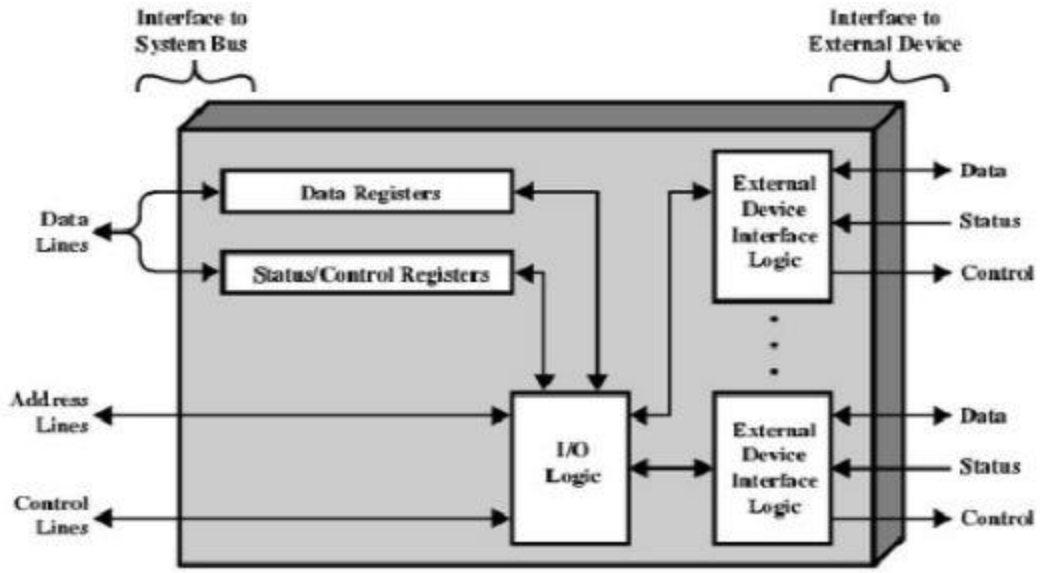


Figure 8

General functions of IO modules can be listed as follows:

1 **Processor communication** -- this involves the following tasks

a. exchange of data between processor and I/O module.
b. command decoding - I/O module accepts commands sent from the processor. E.g., the I/O module for a disk drive may accept the following commands from the processor: READ SECTOR, WRITE SECTOR, SEEK track, etc.
c. status reporting – The device must be able to report its status to the processor, e.g., disk drive busy, ready etc. Status reporting may also involve reporting various errors.
d. Address recognition – Each I/O device has a unique address and the I/O module must recognize this address.

2. **Device communication** – The I/O module must be able to perform device communication such as status reporting.

3. **Control & timing** – The I/O module must be able to co-ordinate the flow of data between the internal resources (such as processor, memory) and external devices.

4. **Data buffering** – This is necessary as there is a speed mismatch between speed of data transfer between processor and memory and external devices. Data coming from

18. **What is the maximum size of the address space generated in a processor with has 32 bit address?**

The maximum size of the address space generated in a processor with has 32 bit address = 2_{32} = 4 Giga Locations.

19. a) How many 128 x 8 RAM chips are needed to provide a memory capacity of 2048 bytes?
b) How many lines of the address bus must be used to access 2048 bytes of memory? How many of these lines will be common to all chips?
c) How many lines must be decoded for chip select? Specify the size of the decoders.
d.1) if a computer uses RAM chips of 1024 x 1 capacity, how many chips are needed to provide a memory capacity of 1024 bytes?
d.2) How many chips are needed to provide a memory capacity of 16K bytes?

a) 128 x 8 RAM => 128 Bytes (8bits) / chip => Nbr of chips = 2048 / 128 = 16

b)

$2048 = 2^d$, where d is nbr of address lines => d = 11 We have 16 chips => 4 bits for chip select Number of common address lines is 11 - 4 = 7 Summary: 4 bits (MSB) to select the correct chip and 7 bits (LSB) to select the memory location inside the selected chip.

c)We need 4 x 16-line decoder

d.1) Memory size is 1024 bytes = 8 x 1024 x 1 RAM => 8 chips

d.2) 16K bytes = 16 x 1024 x 8 => 128 chips.

20. You have RAMs 128 X8 and ROM 512 X 8. You need to make RAM of 512 X 8 RAM. Design memory organisation for the given scenario. What can be the total memory accessed in this organization.

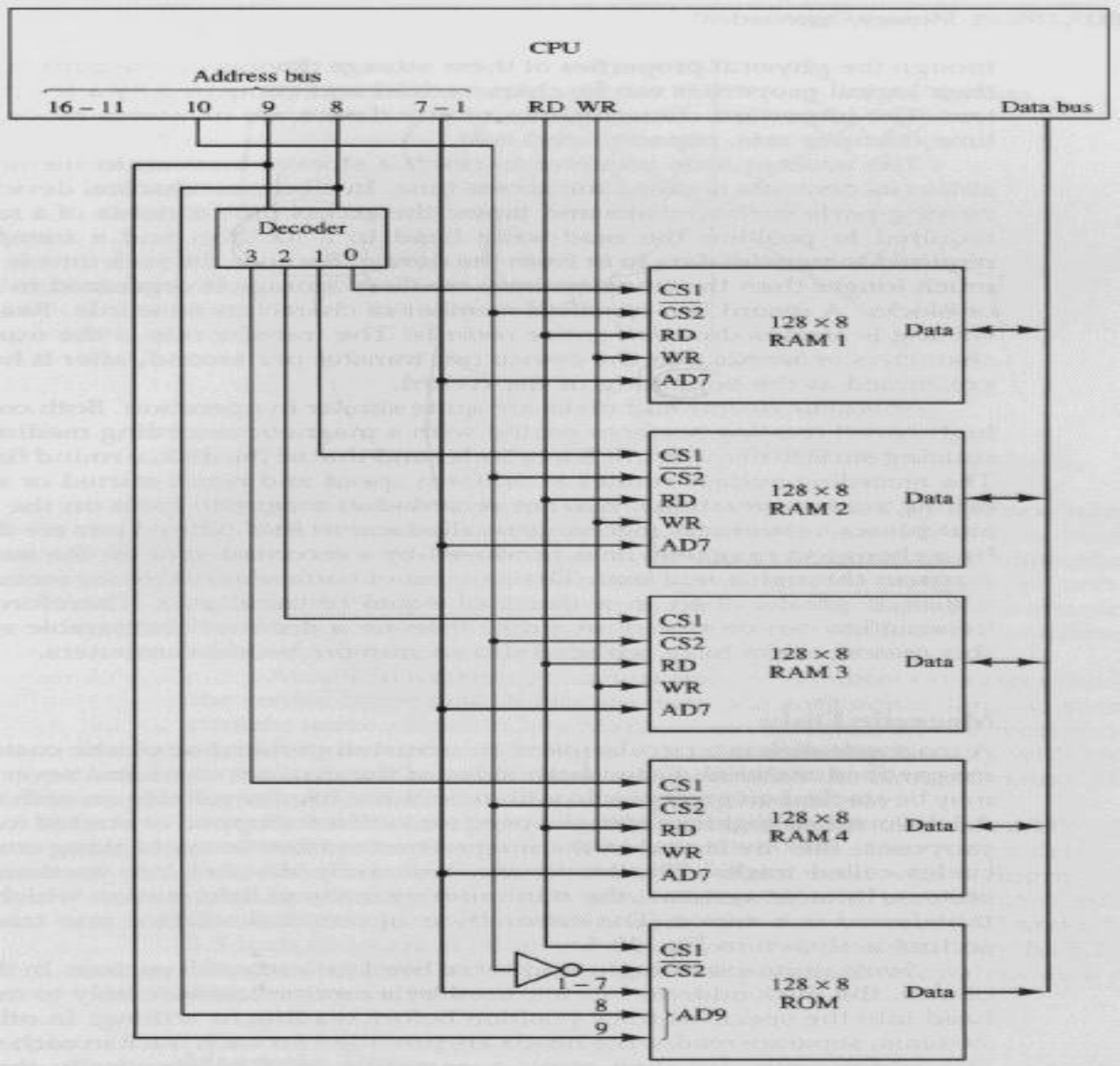


Figure 12-4 Memory connection to the CPU.

21. **An application program in a computer with cache uses 1400 instruction fetch from cache and 100 from main memory. What is hit ratio?**

Hit Ratio = No. of Instruction fetch from cache / Total no. of instructions
= 1400 / 1500 = 93.333%

22. Consider a two level cache with access time 5 nsec and 80 nsec respectively. If the hit ratio are 95% and 75% respectively in the two caches and main memory access time is 250 nsec. What is the effective access time? (effective access time =H1T1+H2(1-H1)T2+…….

Access Time = H1T1 + H2 (1 – H1)T2 + (1 – H1)(1 – H2)T3
= .95(5) + (1 - .95).75 X 80 +(1-.95) (1-.75)250
= 4.75 + 3.00 + 3.125 = 10.875 nsec

21. Let's say that we have two levels of cache, backed by DRAM:
- L1 cache costs 1 cycle to access and has miss rate of 10%
- L2 cache costs 10 cycles to access and has miss rate of 2%
- DRAM costs 80 cycles to access (and has miss rate of 0%)
Then what would be average memory access time (AMAT)?

OLD questions:

1. Consider a 386 system and it has 40 address lines, calculate the total memory it can address. Design a logic circuit to address three 1024 MB RAMs and two 512 MB RAMs .
2. Why RAID system is required? Explain its principles, standard levels, and implementation in detail.
3. Draw the block diagram of IO module and explain each blocks. Explain the IO module decisions with suitable examples.
4. What is a DMA operation? Explain different DMA configurations with suitable block diagrams.
5. Differentiate between static and dynamic RAMs with suitable circuit and explain it with their applications.
6. Draw the block diagram of direct mapping cache organization and explain it. Why cache is required in the computer?
7. Draw the block diagram of high performace bus and explain it. Why do we require high performance bus?
8. Draw the block diagram of typical Cache Organization and explain it. How can you apply Direct Mapping technique in the Cache Organization?